UPFS

OFFICIAL DOCUMENTATION MANUAL

V1.0.0 · UNITY ASSET STORE EDITION

Ultimate Progression Framework System

The Complete Progression Foundation for Unity

A complete, data-driven foundation for creating upgrades, currencies, economies, and progression-focused game systems through professional ScriptableObject workflows.

Publisher: bcyly Labs

Asset Store ID: 99151 · Support: bcylylabs@gmail.com

Unity 2021.3+ / 2022.3+ / 6.0+

ScriptableObject Data Framework

Documentation Outline

Complete structural index and reference roadmap for the UPFS package.

START HERE

System Overview & Core Philosophy ..... OVERVIEW

Upgrade ScriptableObject ..... CORE MANUAL

Installation & Requirements ..... CORE MANUAL

Quick Start ..... CORE MANUAL

CORE COMPONENTS

Currency ScriptableObject ..... CORE COMPONENTS

Upgrade Economy Handler ..... CORE COMPONENTS

Upgrade Info Provider ..... CORE COMPONENTS

Listeners & Events ..... CORE COMPONENTS

EDITOR & WORKFLOWS

Framework Settings ..... EDITOR & WORKFLOWS

Saving & Runtime State ..... EDITOR & WORKFLOWS

Connecting Components ..... EDITOR & WORKFLOWS

Data Generation & Distributions EDITOR & WORKFLOWS

Common Workflows ..... EDITOR & WORKFLOWS

REFERENCE & SUPPORT

Troubleshooting & Validation REFERENCE & SUPPORT

FAQ & Support ..... REFERENCE & SUPPORT

Upgrade API Reference ..... API REFERENCE

Currency API Reference ..... API REFERENCE

Economy & Info API Reference ..... API REFERENCE

FRAMEWORK OVERVIEW

Ultimate Progression Framework System

The Complete Progression Foundation for Unity

A modular, data-driven foundation for creating upgrades, currencies, economies and progression-focused game systems through professional ScriptableObject workflows.

Data-Driven Design

Progression rules, scaling math, cost curves, and milestones reside in authoritative ScriptableObjects rather than rigid hardcoded logic.

Modular Architecture

Use Upgrade SO on its own or seamlessly pair it with Economy Handler, Currency SO, Info Provider, and Event Listeners.

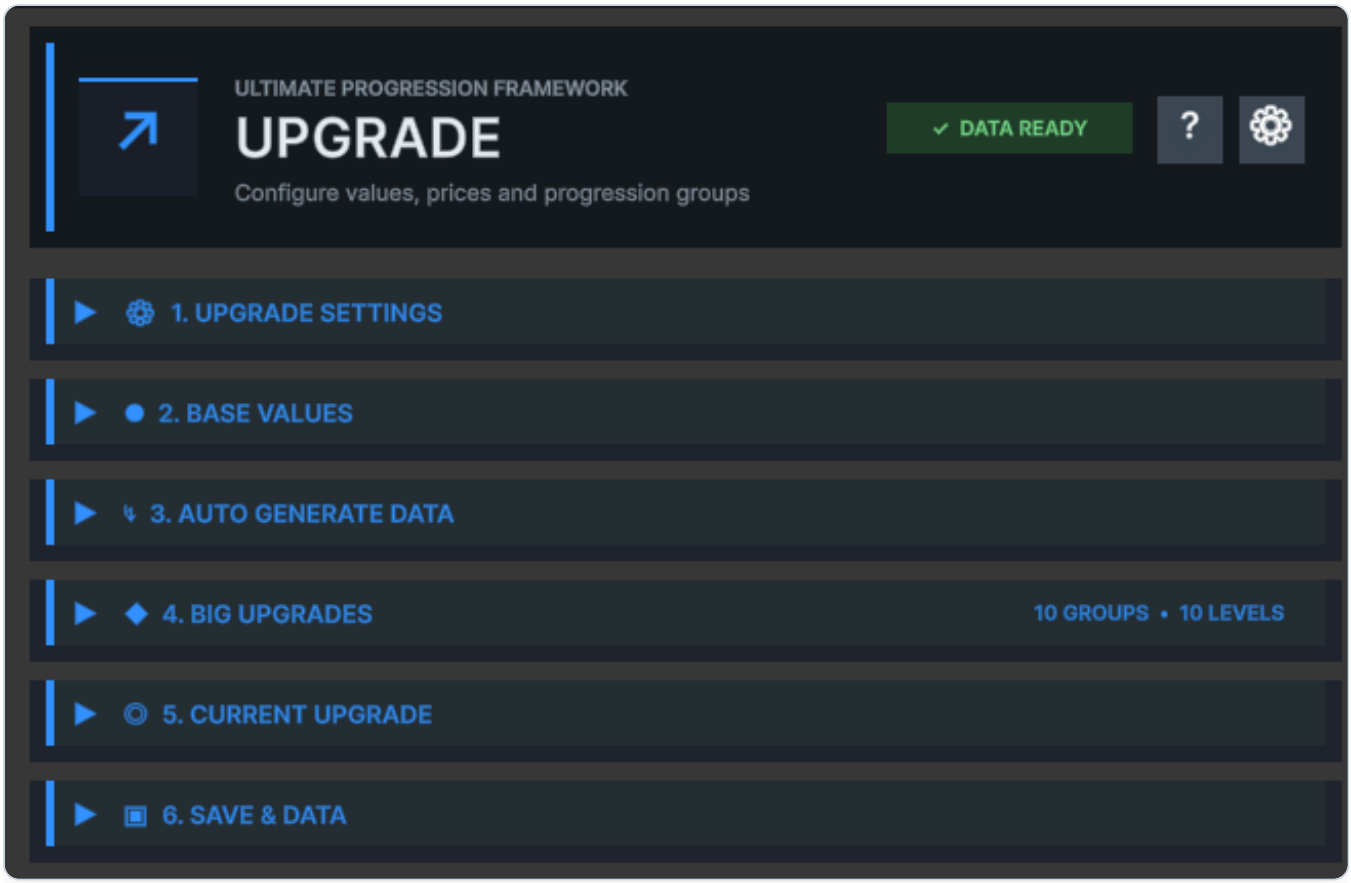
Clean Runtime APIs

Deterministic purchases, queryable balances, safe persistence serialization, and event hooks for UI decoupling.

FOUNDATION COMPONENT

The Upgrade ScriptableObject Central Core

Every progression track in UPFS begins with the Upgrade ScriptableObject. It defines whether upgrades follow simple linear sequences or grouped Big/Small milestone hierarchies. It computes dynamic level curves across 9 math distributions (Linear, Exponential, SmoothStep, Custom Animation Curves) and maintains runtime levels, values, and completion state.

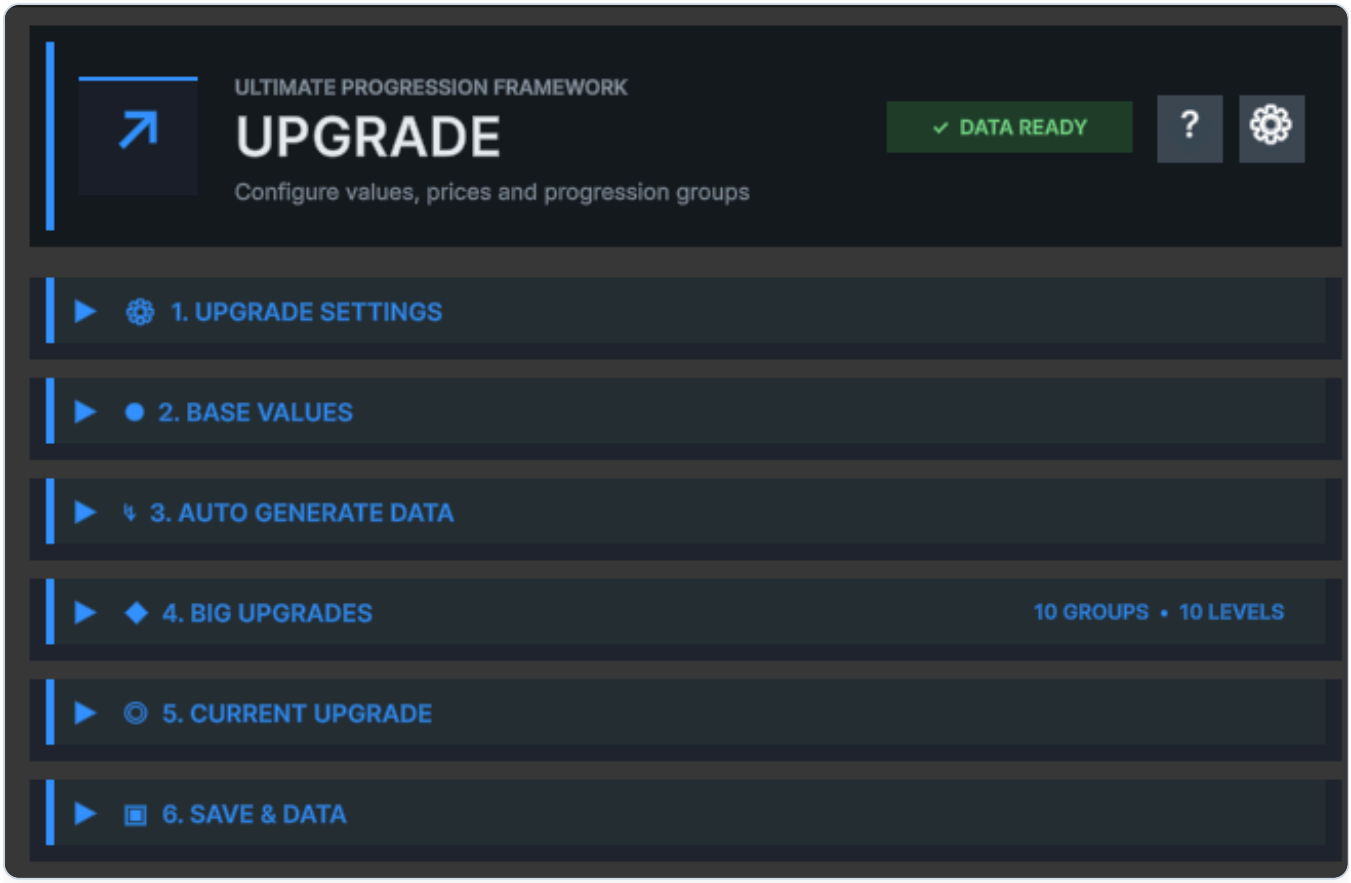


Authoritative Upgrade SO Inspector providing real-time progression tuning and verification.

MODULE 01 CORE MANUAL MAIN COMPONENT

Upgrade ScriptableObject

The Upgrade ScriptableObject is the foundation of UPFS. It defines progression structure, levels, values, prices, milestones, distributions and runtime state in one reusable data asset.



The Upgrade Inspector organizes the complete progression workflow into six focused sections.

BEFORE YOU BEGIN

The progression model

UPFS organizes progression into Big Upgrade groups containing sequential Small Upgrades.

A Small Upgrade represents one purchasable or unlockable progression step. It stores a level, gameplay value, price and completion state.

A Big Upgrade groups related Small Upgrades into a milestone. Groups make long progressions easier to author, display and react to without changing the sequential runtime model.

ELEMENT	PURPOSE	EXAMPLE
Upgrade SO	Owns the complete progression	Weapon Power
Big Upgrade	Groups a progression milestone	Steel Tier
Small Upgrade	Represents one sequential step	Level 4 → Level 5

Critical Requirement: Sequential by design

PerformUpgrade advances the first incomplete Small Upgrade. UPFS does not skip incomplete steps inside the standard Upgrade SO workflow.

Create an Upgrade asset

Create the main progression asset directly from Unity's Project window.

01 **Open the Create menu**

Right-click inside the Project window.

02 **Choose the UPFS asset**

Select Bcyly Labs › Ultimate Progression Framework › Upgrade.

03 **Name the asset**

Use a stable, descriptive name such as WeaponPower_Upgrade.

04 **Configure the structure**

Open Upgrade Settings, then use Auto Generate Data or build the list manually.

Best Practice Tip: Use one responsibility per asset

Create separate Upgrade assets for independent progressions such as Weapon Power, Storage Capacity and Player Level.

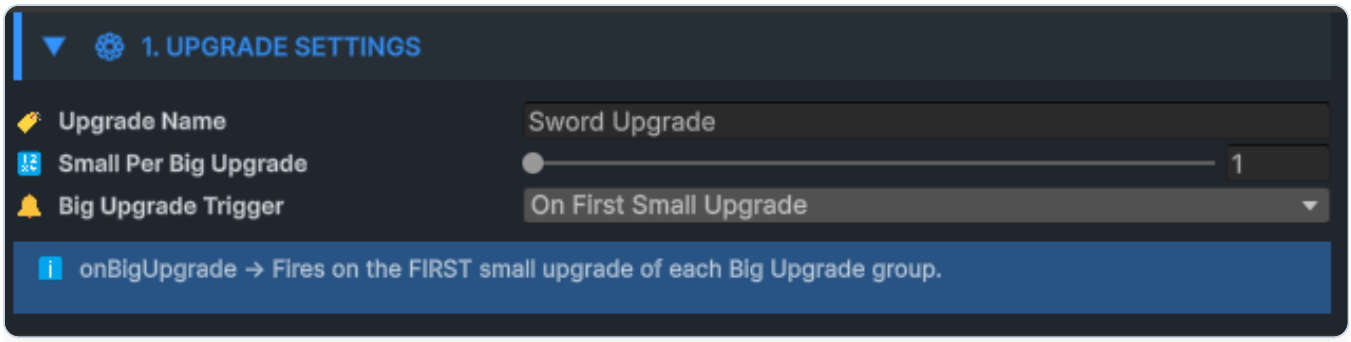
INSPECTOR SECTION

1. Upgrade Settings

Defines the identity, group structure and Big Upgrade milestone trigger behavior.

Upgrade Name identifies the progression at runtime and is currently used as part of its PlayerPrefs save key. Small Upgrades Per Big Upgrade establishes the default group size used by the data generator.

Big Upgrade Trigger Mode determines whether a Big Upgrade event is raised when a group begins or when its final Small Upgrade is completed.



Upgrade Settings controls progression identity, default group size and milestone timing.

CONTROL	PURPOSE	GUIDANCE
Upgrade Name	Human-readable progression identity and save-key input	Keep it unique and avoid renaming after release
Small Upgrades Per Big Upgrade	Default number of steps in each generated group	Choose a size that matches the intended milestone rhythm
On First Small Upgrade	Raises the Big Upgrade event when a group starts	Useful for entering a new tier
On Last Small Upgrade	Raises the Big Upgrade event when a group completes	Useful for milestone rewards

Important Notice / Warning: Save-key stability

Changing Upgrade Name changes the current PlayerPrefs key. Finalize stable names before shipping persistent player data.

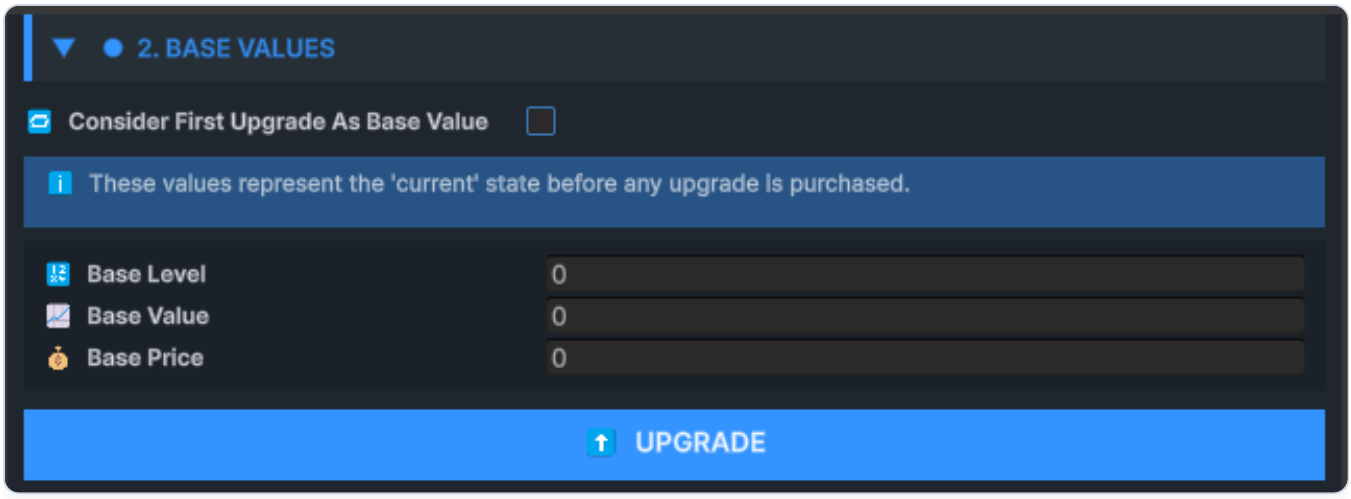
INSPECTOR SECTION

2. Base Values

Defines the state returned before the first standard upgrade has been completed.

Base Values provide the starting level, gameplay value and price when the progression has no purchased Small Upgrade. This makes the current-state API useful from the first frame.

When Consider First Upgrade As Base Value is enabled, the first Small Upgrade is silently marked as the base state. Gameplay events are not fired for this automatic initialization.



Base Values establish the authored starting state before normal progression begins.

Separate base values

Use the dedicated Base Values fields when level zero or an unpurchased state exists outside the generated upgrade list.

First upgrade as base

Use the first generated entry as the initial state when every meaningful state should live inside one continuous list.

Information: No gameplay event on initialization

The first entry is applied as a base without invoking Small Upgrade, Big Upgrade or Any Upgrade events.

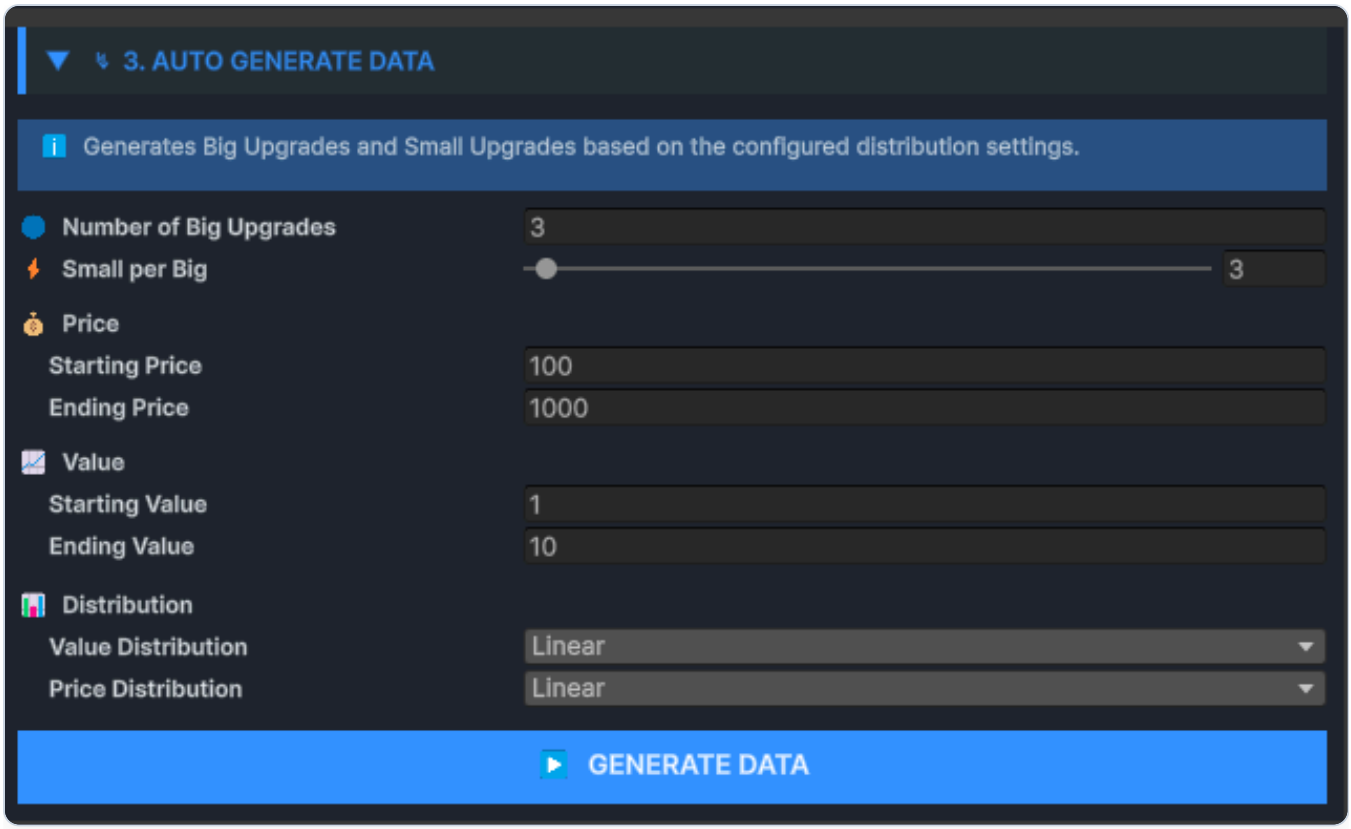
INSPECTOR SECTION

3. Auto Generate Data

Builds a complete progression from structure, value, price and distribution settings.

The generator is the fastest way to author consistent upgrade data. Define the number of groups and steps, then choose start/end ranges for gameplay values and prices.

Value and price distributions are independent. A progression can use smooth value growth with aggressive exponential cost scaling, or any other supported pairing.



The generator creates nested upgrade data and applies independent value and price distributions.

DISTRIBUTION	BEHAVIOR	TYPICAL USE
Linear	Even change from start to end	Predictable level progression
Exponential / Ease In	Slower early, faster late growth	Long-term price scaling
Logarithmic / Ease Out	Fast early gains that gradually flatten	Early player acceleration
Ease In Out	Smooth S-curve	Balanced middle-focused progression
Random	Value selected from configured bounds	Authored variation
Step	Keeps the starting value	Flat values with separate cost growth
Custom Curve	Evaluates an AnimationCurve	Fully authored progression shape

Important Notice / Warning: Regeneration replaces authored structure

Review the Inspector confirmation before generating over existing Big and Small Upgrade data.

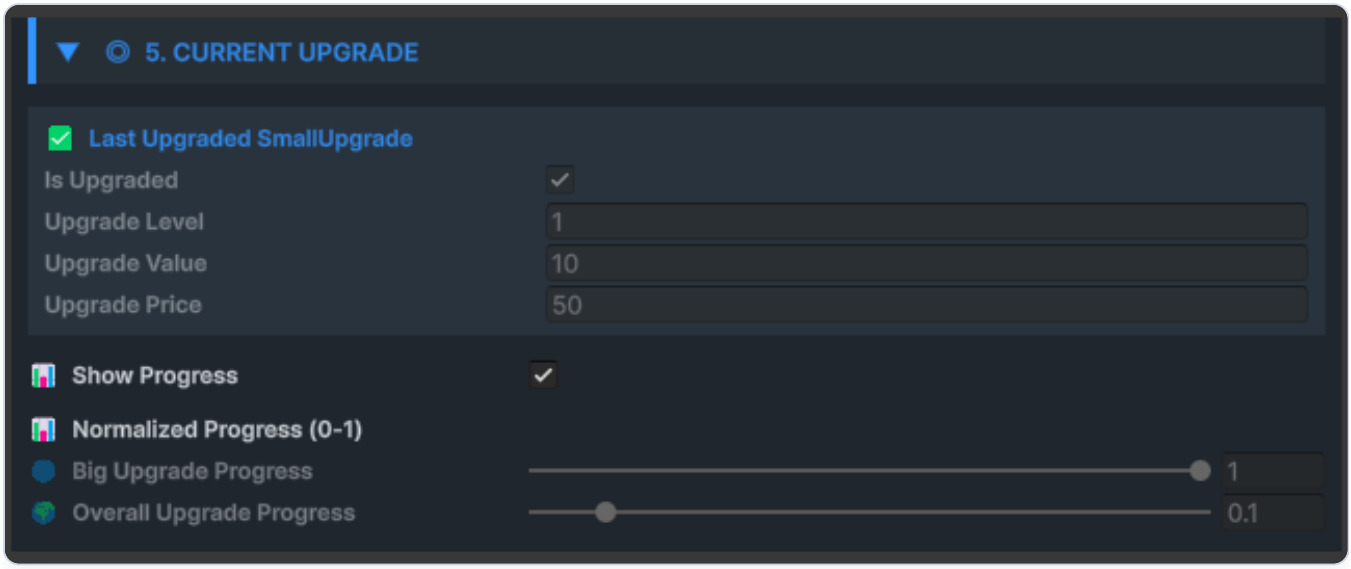
INSPECTOR SECTION

4. Current Upgrade

Displays and tests the live progression state without searching through nested data.

Current Upgrade provides an immediate view of the active level, value and progression status. It is useful while balancing data and while testing persistence in Play Mode.

The runtime API exposes the same state through GetCurrentUpgrade, GetNextUpgrade, GetOverallUpgradeProgress and GetCurrentBigUpgradeProgress.



Current Upgrade summarizes active data and normalized progression at a glance.

- Current Small Upgrade level and gameplay value
- Next available upgrade and cost
- Progress inside the active Big Upgrade group
- Overall progress across the complete asset
- Fully upgraded state

> **Best Practice Tip: Testing workflow**
> Use a copy of production data when manually advancing progression in the Inspector during development.

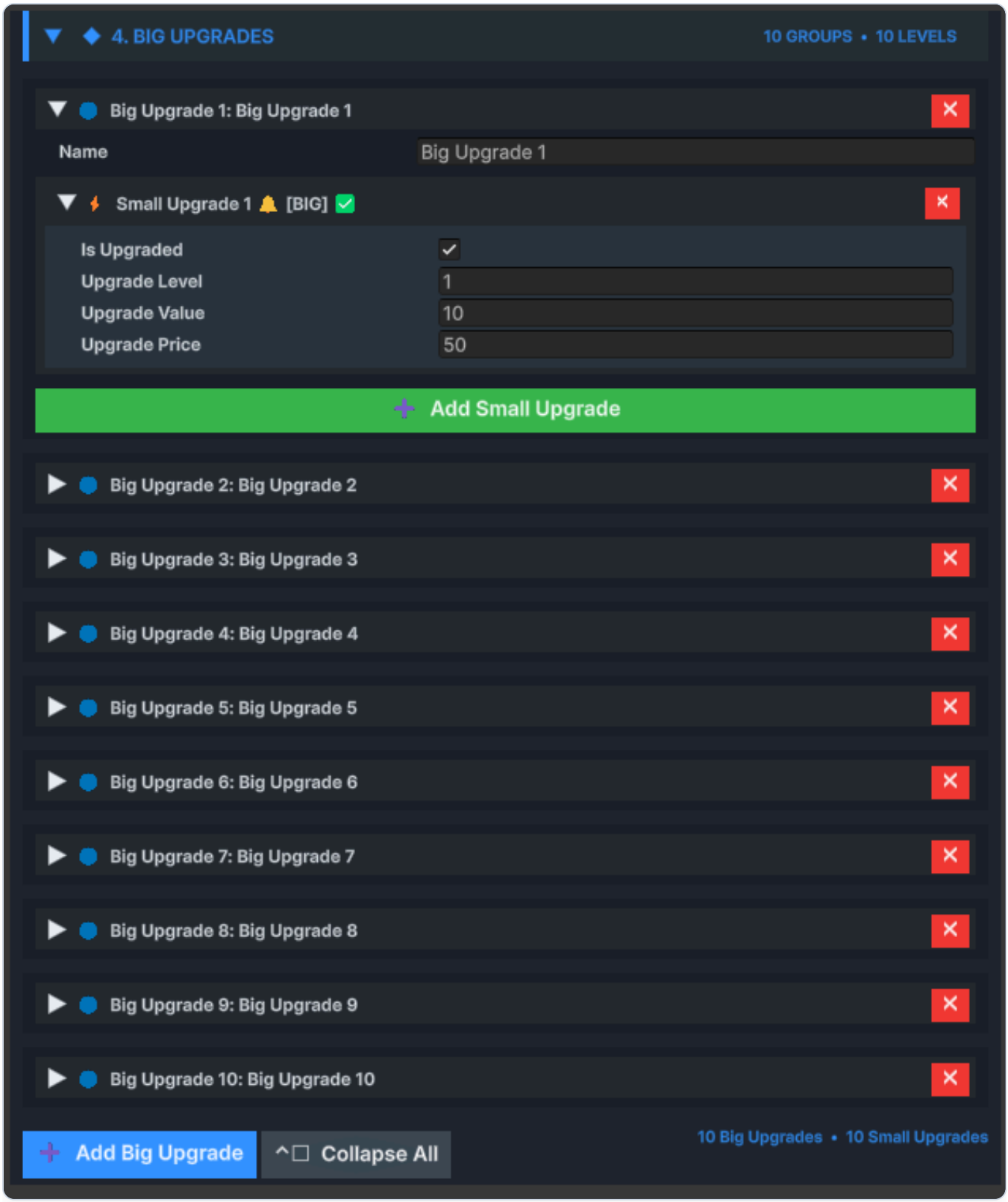
INSPECTOR SECTION

5. Big Upgrades

Provides direct control over every group and individual progression step.

Each Big Upgrade contains a name and a nested list of Small Upgrades. Each Small Upgrade stores completion state, level, value and the native upgrade price.

Use this section to review generated data, fine-tune individual entries, add special milestone values or manually construct a progression that does not follow one mathematical curve.



Big Upgrades exposes the complete nested progression with focused add, edit and delete controls.

SMALL UPGRADE FIELD	RUNTIME MEANING
Is Upgraded	Whether this step has been completed
Upgrade Level	Level identifier presented to gameplay and UI
Upgrade Value	Gameplay value associated with this state
Upgrade Price	Native single-price value stored by Upgrade SO

> **Critical Requirement: Use Economy Handler for multi-currency costs**
> Upgrade Price is the native integer price. Connect an Upgrade Economy Handler when the progression requires multiple currencies or payment modes.

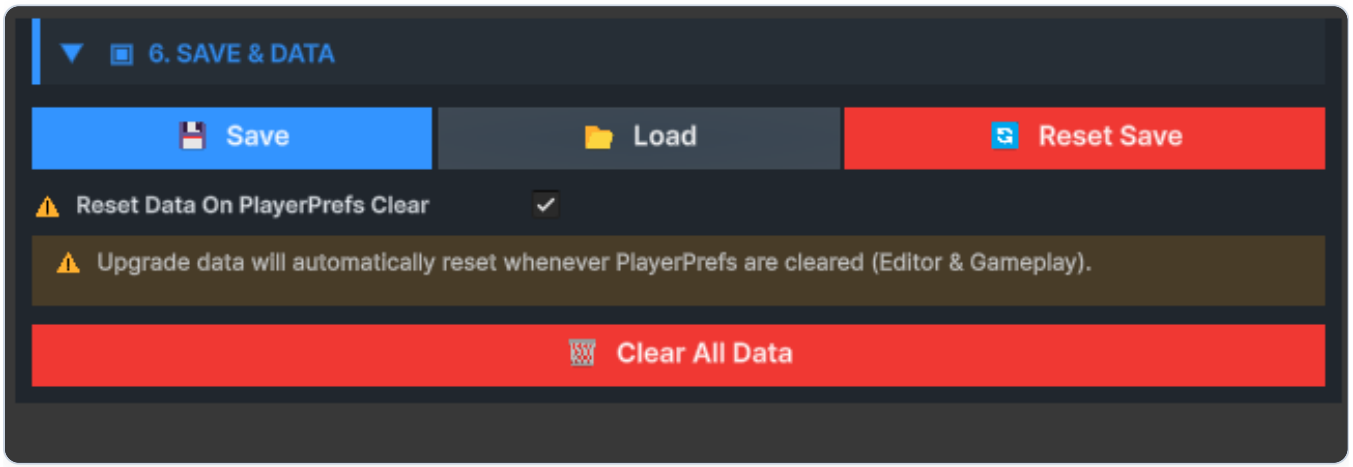
INSPECTOR SECTION

6. Save & Data

Controls PlayerPrefs persistence, loading, resets and development-time data actions.

Upgrade SO serializes its Big Upgrade data to JSON and stores it in PlayerPrefs. PerformUpgrade automatically saves after a successful progression step.

Load restores the saved structure and completion state. Reset deletes the asset's save key, clears completed states and reapplies the first entry when it is configured as the base value.



Save & Data provides explicit development controls around the automatic persistence lifecycle.

ACTION / SETTING	RESULT
Save	Writes current Big Upgrade data to PlayerPrefs
Load	Replaces runtime data with the stored JSON data
Reset Save	Deletes the key and clears completion state
Reset on PlayerPrefs Cleared	Clears progression when no save key is found

Important Notice / Warning: Current persistence scope

UPFS includes PlayerPrefs persistence for its core assets. Cloud storage and third-party save integrations are not built-in features of this version.

Runtime upgrade flow

One successful PerformUpgrade call updates state, raises events and saves the progression.

01 **Find next step**

UPFS scans Big Upgrades in order and selects the first incomplete Small Upgrade.

02 **Complete state**

The selected Small Upgrade is marked as upgraded.

03 **Build event context**

Detailed data includes Small and Big Upgrade references plus local and global indices.

04 **Raise events**

Any Upgrade and detailed events fire, followed by the configured Small or Big milestone event.

05 **Check completion**

The Fully Upgraded event fires when no next entry remains.

06 **Persist**

The updated data is saved to PlayerPrefs.

CSHARP

```
public class WeaponUpgradeController : MonoBehaviour
{
    [SerializeField] private Upgrade weaponPower;

    public void UpgradePower()
    {
        if (!weaponPower.IsFullyUpgraded())
            weaponPower.PerformUpgrade();
    }
}
```

Essential Upgrade API

The most commonly used runtime calls for reading and advancing progression.

MEMBER	RETURNS	PURPOSE
PerformUpgrade()	bool	Completes the next sequential Small Upgrade
GetCurrentUpgrade()	SmallUpgrade	Returns the current state or base values
GetNextUpgrade()	SmallUpgrade	Returns the next incomplete step
GetSmallUpgradeByIndex(index)	SmallUpgrade	Reads an entry by zero-based global index
GetUpgradedSmallCount()	int	Returns the sequential completed count
GetOverallUpgradeProgress()	float	Returns normalized total progress
IsFullyUpgraded()	bool	Checks whether all entries are complete

Information: Complete reference

See Upgrade API Reference for all methods, events, nested data types and return behavior.

MODULE 02 CORE MANUAL

Installation & Requirements

UPFS is delivered as a Unity Asset Store package. Import the package, allow Unity to compile its runtime and Editor assemblies, then create the Upgrade ScriptableObject that will own your first progression.

Project requirements

The framework uses standard Unity ScriptableObjects, Editor scripting, UnityEvents and PlayerPrefs.

- A Unity project compatible with the version declared on the current Asset Store listing
- No third-party Inspector or serialization dependency
- Write access to PlayerPrefs when using the included persistence behavior
- Unity UI or UI Toolkit only when your own runtime interface requires it

Information: Render-pipeline independent

The core framework is data and Editor tooling. It does not depend on Built-in, URP or HDRP rendering features.

Import the package

01 Open Package Manager

Choose Window › Package Manager and open My Assets.

02 Find UPFS

Select Ultimate Progression Framework System and download the current release.

03 Import

Import the framework folders and wait for Unity to finish compiling.

04 Confirm the Create menu

Right-click in the Project window and locate Bcyly Labs › Ultimate Progression Framework.

Critical Requirement: Resolve compile errors first

Unity custom Inspectors load only after the project compiles successfully. Existing errors elsewhere in the project can prevent the UPFS Inspector from appearing.

Core package structure

Runtime components are separated from professional Editor tooling.

AREA	CONTENTS	BUILD INCLUSION
Upgrade	Upgrade data and UpgradeListener	Runtime
Currency	Currency data and CurrencyListener	Runtime
Economy	UpgradeEconomyHandler and listener	Runtime
Info	UpgradeInfoProvider metadata	Runtime
Editor	Inspectors, settings and PlayerPrefs tools	Editor only

Verify the installation

CSHARP

```
using bcylyLabs.UltimateProgressionFramework;
using UnityEngine;

public class UpfsInstallCheck : MonoBehaviour
{
    [SerializeField] private Upgrade upgrade;
}
```

Best Practice Tip: Next step

Continue to Quick Start to create an Upgrade, Currency and Economy Handler and complete the first purchase.

MODULE 03 CORE MANUAL MUST READ

Quick Start

This workflow creates one upgrade progression, one currency and one Economy Handler. It is the shortest path from a clean project to a successful runtime purchase.

1. Create the main Upgrade SO

01 Create Upgrade
Choose Create › Bcyly Labs › Ultimate Progression Framework › Upgrade.

02 Set identity
Name it WeaponPower_Upgrade and enter a unique Upgrade Name.

03 Choose base behavior
Set explicit Base Values or enable Consider First Upgrade As Base Value.

04 Generate data
Use Auto Generate Data to create the desired Big and Small Upgrade structure.

2. Create a Currency SO

01 Create Currency
Choose Create › Bcyly Labs › Ultimate Progression Framework › Currency.

02 Assign a unique ID
Use an ID not used by any other Currency asset.

03 Set name and type
Enter Gold and choose Int for whole-number balances.

04 Set default value
Use a starting balance large enough to test the first purchase.

3. Configure an Economy Handler

01 **Create handler**

Choose Create › Bcyly Labs › Ultimate Progression Framework › Upgrade Economy Handler.

02 **Assign Upgrade Source**

Drag WeaponPower_Upgrade into Upgrade Source & Sync, then fetch its data.

03 **Add Gold costs**

Create a Gold currency definition and distribute its starting and ending costs.

04 **Build the wallet**

Add the Gold Currency SO to Wallet and choose a Payment Mode.

Critical Requirement: Names must match

The Economy Handler matches a wallet Currency to cost data by Currency Name. Use the same name in the Currency SO and economy definition.

4. Purchase at runtime

CSHARP

```
using bcylyLabs.UltimateProgressionFramework;
using UnityEngine;

public class UpgradeButton : MonoBehaviour
{
    [SerializeField] private UpgradeEconomyHandler economy;

    public void Purchase()
    {
        UpgradeEconomyHandler.PurchaseResult result =
            economy.PurchaseNextUpgrade();

        if (!result.success)
            Debug.LogWarning(result.failReason);
    }
}
```

Best Practice Tip: Player Choice mode

Call GetAffordableOptionsForNext, present those options in your UI, then pass the selected Currency to PurchaseWithCurrency.

5. Verify the result

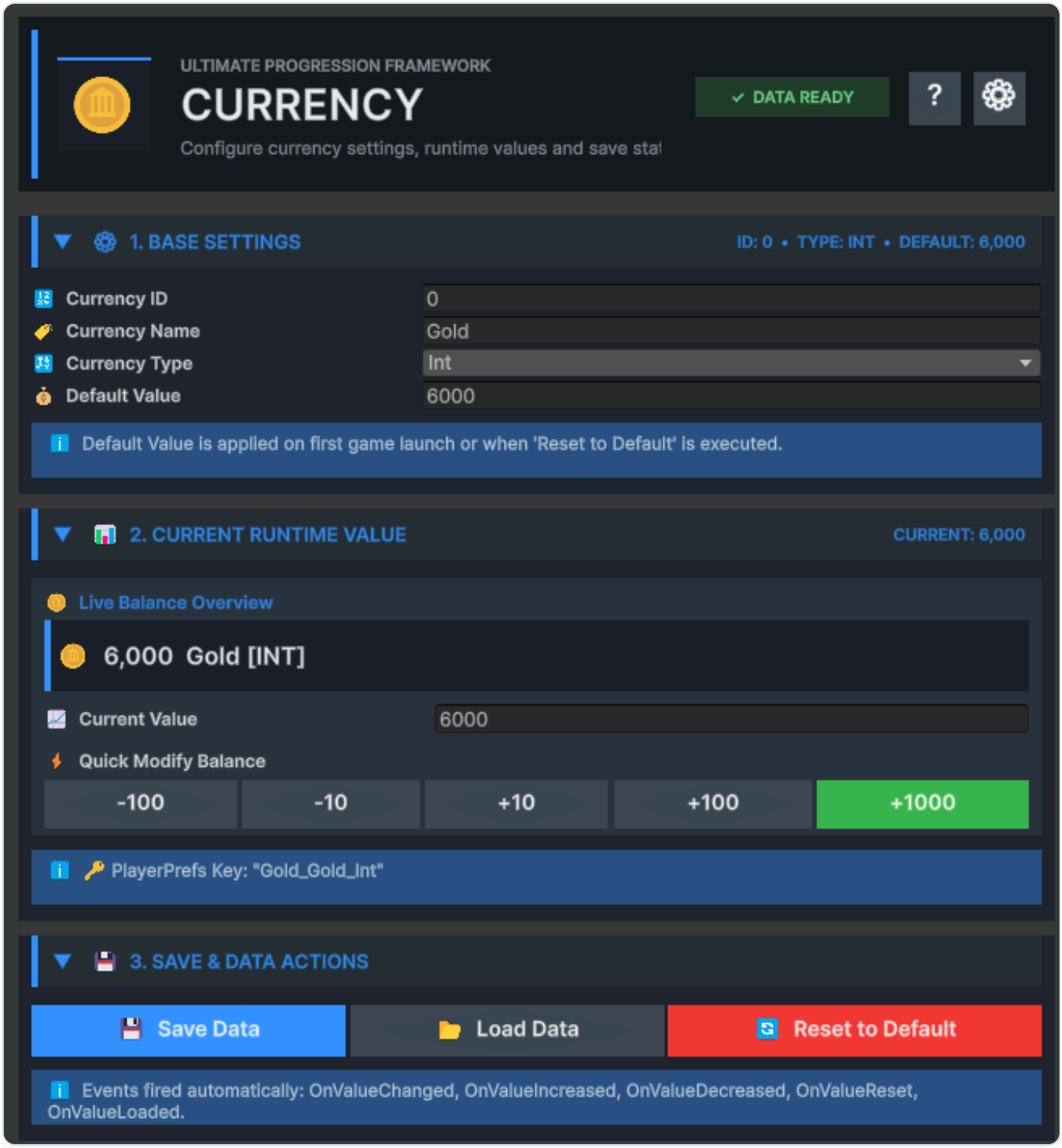
- The Currency balance decreases by the selected cost
- The next Small Upgrade becomes completed

- Upgrade and Economy events are raised
- The Upgrade and Currency data are saved
- The next cost and progress queries reflect the new state

MODULE 04 CORE COMPONENTS

Currency ScriptableObject

Currency SO stores one shared balance that can be referenced throughout the project. It supports integer or floating-point values, automatic PlayerPrefs persistence and focused change events.



The Currency Inspector separates identity, live balance and persistence actions into three focused sections.

Create a Currency asset

01 **Open Create menu**
Right-click in the Project window.

02 **Choose Currency**
Select Bcyly Labs › Ultimate Progression Framework › Currency.

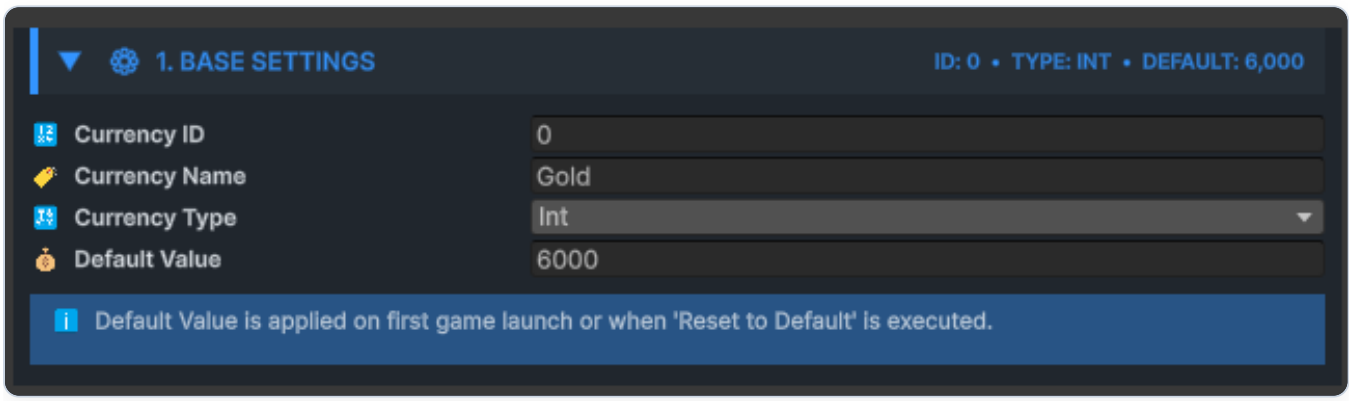
03 **Assign a unique ID**
Currency IDs are used by economy APIs and must not be duplicated.

04 **Set a stable name**
The name participates in persistence keys and economy matching.

INSPECTOR SECTION

1. Base Settings

Defines the currency identity, numeric behavior and first-run balance.



Base Settings provides the stable identity and authored default balance.

FIELD	PURPOSE	GUIDANCE
ID	Integer lookup key for public APIs	Keep unique across Currency assets
Currency Name	Display and economy-matching name	Keep unique and stable
Type	Int rounds values; Float preserves decimals	Choose according to gameplay precision
Default Value	Balance used when no save exists	Set the intended first-run amount

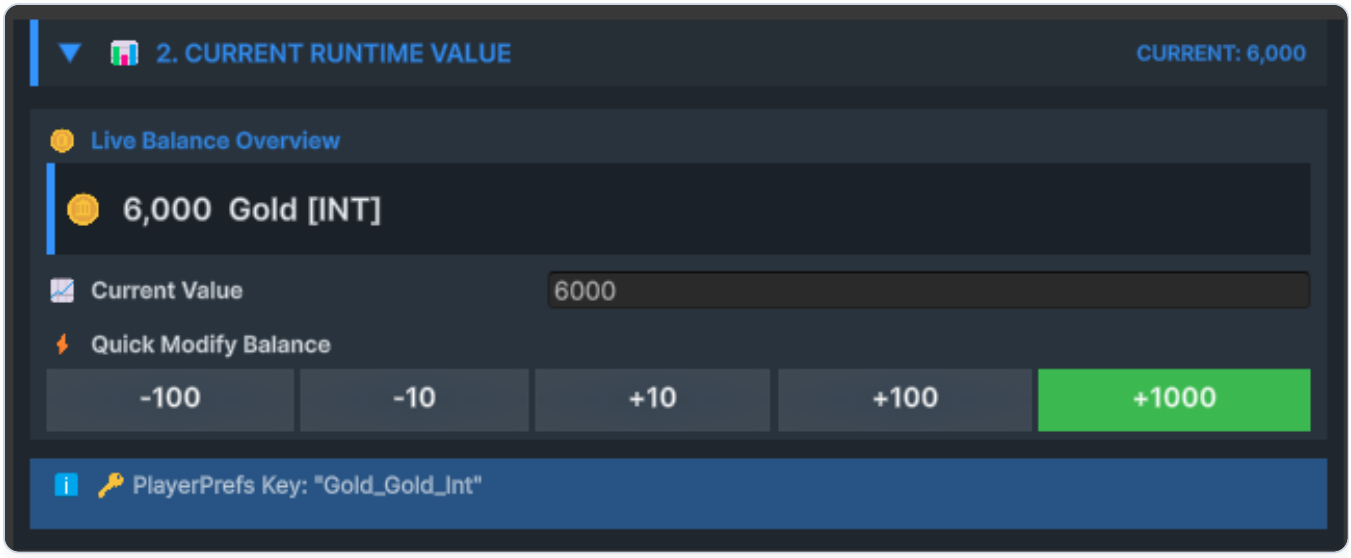
Important Notice / Warning: Type-specific save keys
Changing between Int and Float switches the key format. Treat a post-release type change as a save-data migration.

INSPECTOR SECTION

2. Current Runtime Value

Displays and modifies the live shared balance for development and runtime testing.

GetValue returns the balance after applying the configured numeric type. SetValue replaces it; ModifyValue adds a positive or negative delta. Both operations save immediately and raise change events.



Runtime Value exposes the shared balance and quick modification controls.

```csharp
gold.SetValue(500f);
gold.ModifyValue(75f);   // Gain 75
gold.ModifyValue(-25f);  // Spend 25

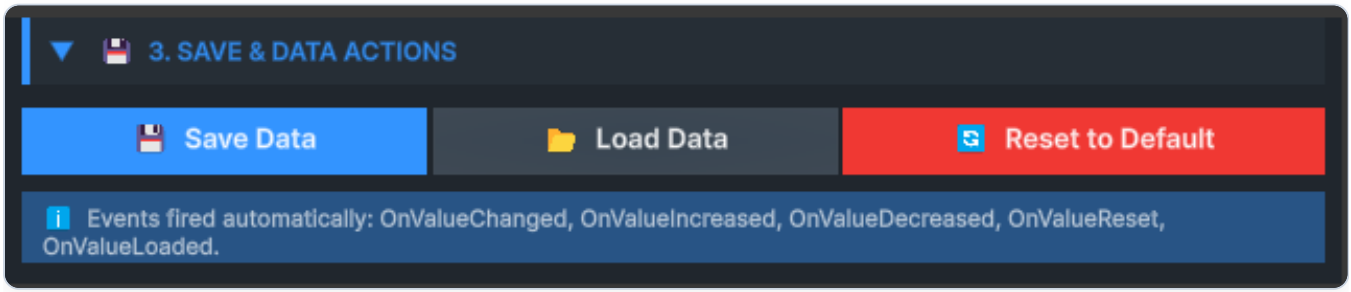
if (gold.HasEnough(100f))
    Debug.Log("Purchase available");
```

> **Information: Shared data asset**
> Every object referencing the same Currency SO reads and modifies the same live balance.

INSPECTOR SECTION

3. Save & Data Actions

Provides manual Save, Load and Reset operations around the automatic persistence behavior.



Save & Data Actions supports explicit development-time persistence testing.

ACTION	BEHAVIOR
Save	Stores the typed current value in PlayerPrefs
Load	Loads the typed key or falls back to Default Value
Reset	Restores Default Value, saves and raises OnValueReset

Best Practice Tip: Listen without scripting

Add CurrencyListener to a GameObject to route balance events into Inspector-configured UnityEvents.

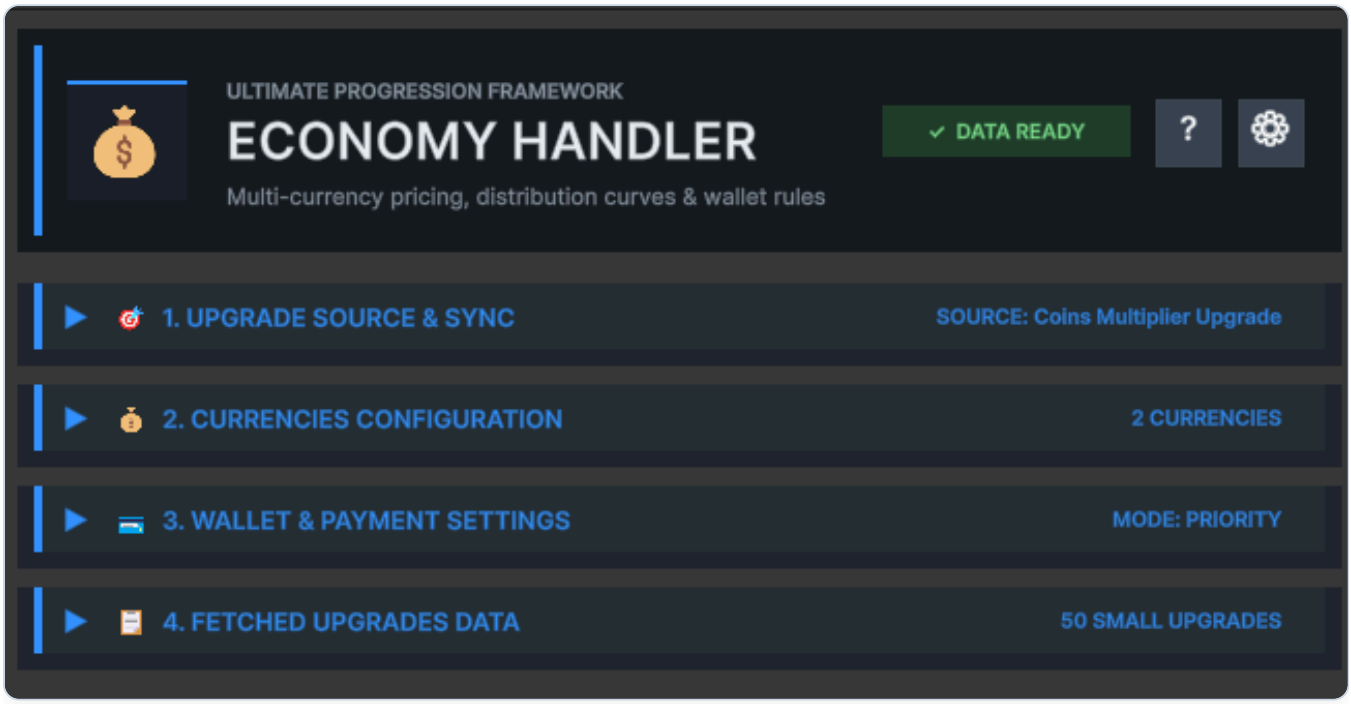
Currency events

EVENT	VALUE PASSED
OnValueChanged	New current balance
OnValueIncreased	Positive amount gained
OnValueDecreased	Positive magnitude deducted
OnValueReset	Reset balance
OnValueLoaded	Loaded balance

MODULE 05 CORE COMPONENTS

Upgrade Economy Handler

Upgrade Economy Handler connects an Upgrade SO to one or more Currency assets. It generates per-level costs, evaluates affordability and executes controlled purchases through three payment modes.



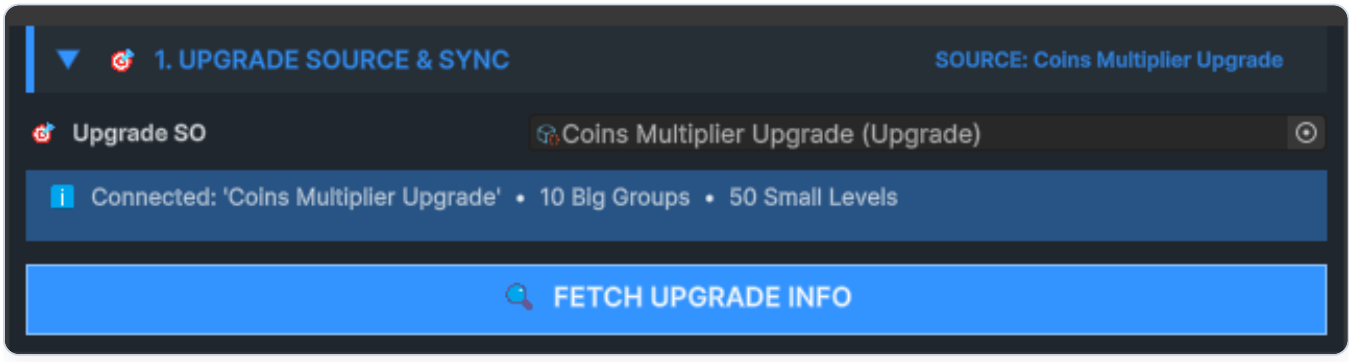
The Economy Inspector moves from source synchronization to cost configuration, wallet behavior and generated data.

INSPECTOR SECTION

1. Upgrade Source & Sync

Links the economy to its authoritative Upgrade SO and copies its current structure.

Fetch Upgrade Info creates a flattened economy entry for every Small Upgrade while retaining its Big and Small indices, level, value and native price.



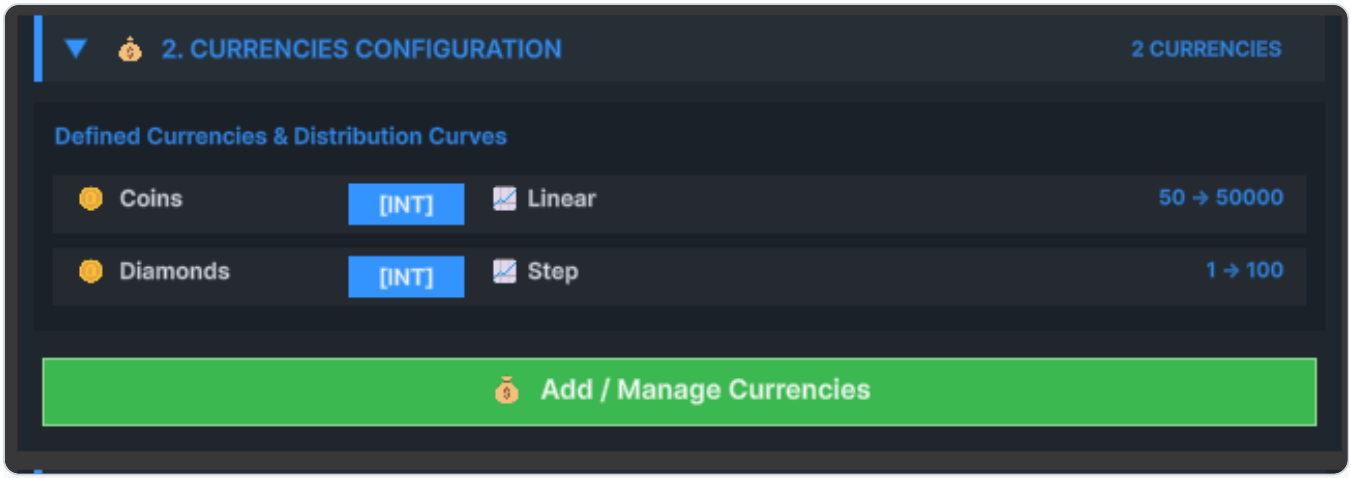
Assign the target Upgrade and fetch whenever its authored structure changes.

> **Important Notice / Warning: Re-fetch after structural edits**
>
> Additions, deletions or ordering changes in Upgrade SO are not automatically reflected in previously fetched economy data unless the relevant auto-fetch preference is enabled.

INSPECTOR SECTION

2. Currencies Configuration

Defines every currency-specific cost series available to this progression.



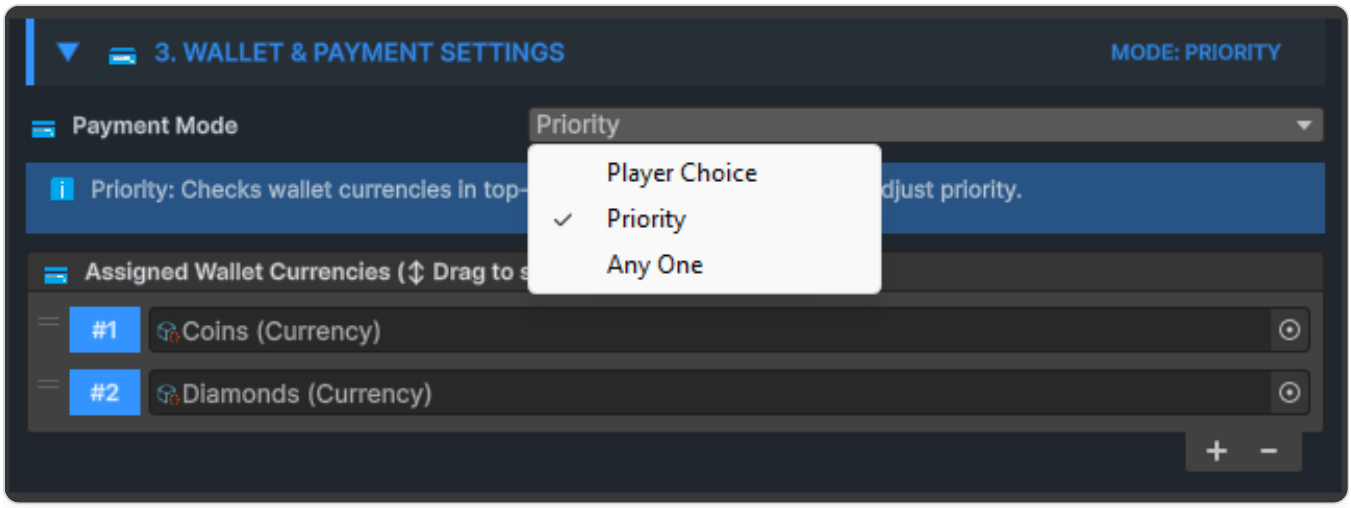
Each definition owns a name, numeric type, range and distribution curve.

SETTING	PURPOSE
Currency Name	Matches generated cost data to a wallet Currency
Int / Float	Rounds or preserves distributed costs
Starting / Ending Value	Defines the cost range
Distribution	Controls the shape of cost growth
Random Min / Max	Bounds Random distribution
Custom Curve	Authors normalized progression manually

INSPECTOR SECTION

3. Wallet & Payment Settings

Defines which Currency assets can pay and how the payer is selected.



Wallet order provides deterministic selection for automatic and priority purchases.

PAYMENT MODE	BEHAVIOR	REQUIRED API
Any One	Uses the first affordable wallet currency with a matching cost	PurchaseNextUpgrade
Priority	Uses wallet order as explicit payment priority	PurchaseNextUpgrade
Player Choice	The game presents valid options and supplies the selected Currency	GetAffordableOptionsForNext + PurchaseWithCurrency

Critical Requirement: Wallet order matters

For automatic selection, place the most preferred Currency first in the Wallet list.

INSPECTOR SECTION

4. Fetched Upgrades Data

Shows the flattened upgrade sequence and generated cost for every currency.



Fetched data makes generated costs reviewable before they are used at runtime.

- Big and Small Upgrade indices
- Level, value and native price
- One generated cost entry per configured currency

- Sequential order used by purchase and Buy Max queries

> **Best Practice Tip: Balance before runtime**
> Review the first, middle and final generated costs to confirm that the selected distribution matches the intended economy curve.

Purchase results and events

Every purchase returns structured success or failure information.

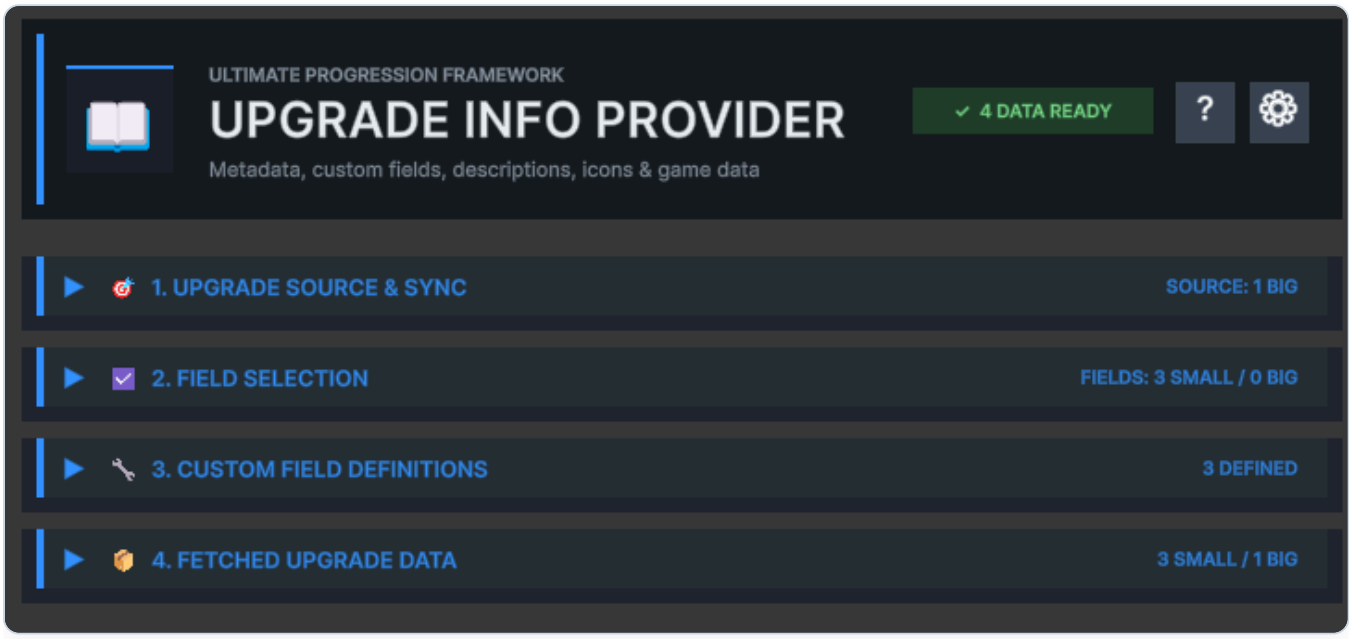
- OnUpgradePurchased
- OnUpgradeFailed
- OnFullyUpgraded

RESULT FIELD	MEANING
success	Whether the transaction completed
failReason	Readable explanation when it did not complete
currencyUsed	Name of the paying Currency
amountDeducted	Final charged amount

MODULE 06 CORE COMPONENTS

Upgrade Info Provider

Upgrade Info Provider transforms Upgrade SO data into a UI-friendly information layer. Select the built-in fields your interface needs, add typed custom fields, and query one consistent source at runtime.



The Info Provider Inspector separates source synchronization, field selection, custom definitions and fetched output.

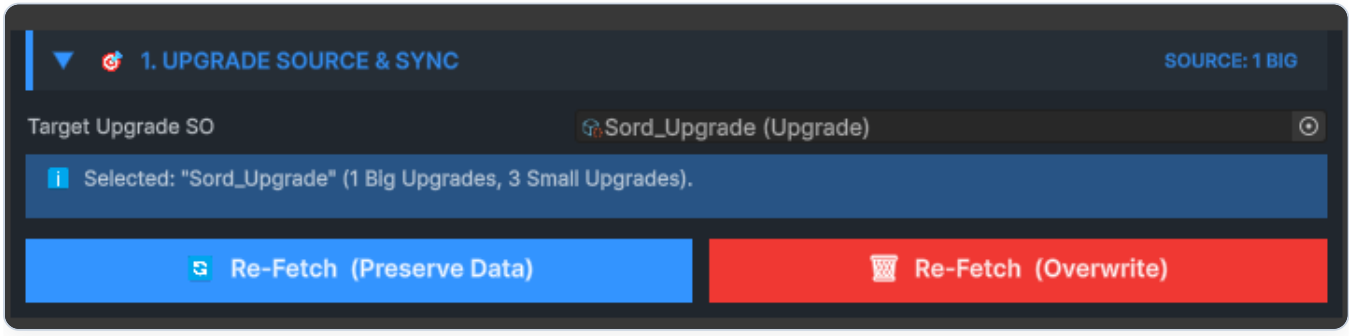
INSPECTOR SECTION

1. Upgrade Source & Sync

Connects the provider to the authoritative Upgrade SO and controls synchronization.

Fetch Upgrade Info reads the nested Big and Small Upgrade structure and creates a presentation record for each step.

Use preserveExisting when refetching after structural edits if existing custom values should remain attached where possible.



Assign the Upgrade source and fetch its current structure before configuring presentation data.

Important Notice / Warning: Refetch after source changes

When groups, levels or ordering change in Upgrade SO, refetch the provider before relying on indexed information at runtime.

INSPECTOR SECTION

2. Field Selection

Chooses which standard Upgrade fields are copied into the information layer.



Enable only the standard fields required by gameplay or UI consumers.

- Big and Small Upgrade names and indices
- Upgrade level, value and native price
- Completion and current-state information
- Fields used by cards, tooltips, previews and selection screens

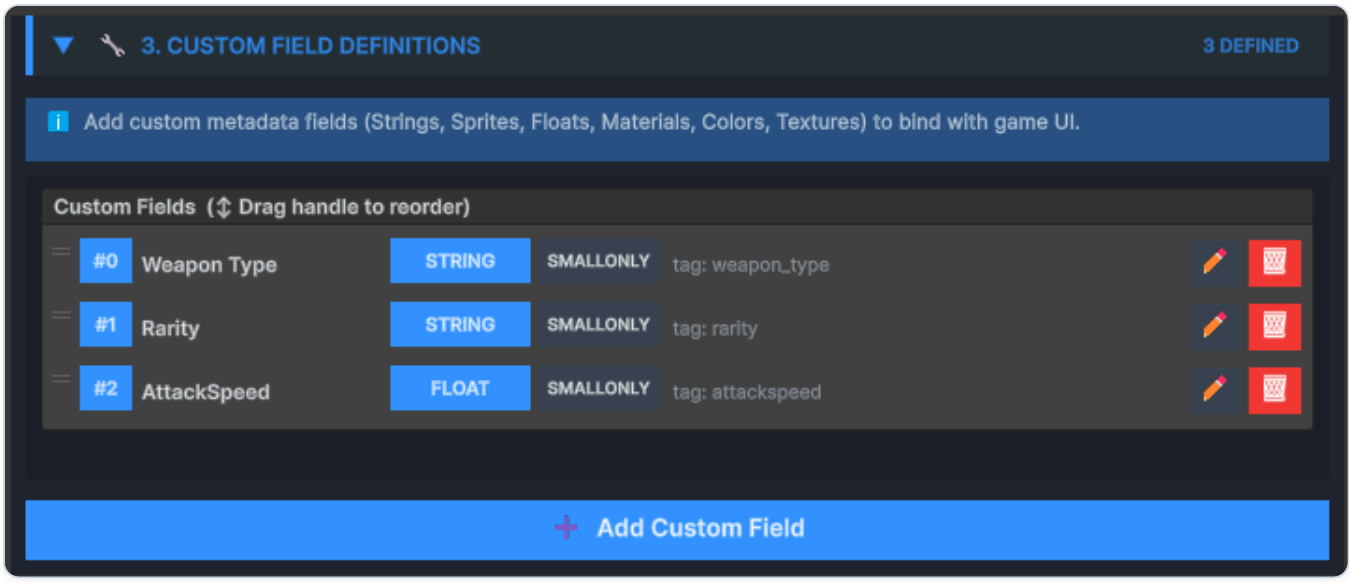
Best Practice Tip: Keep payloads intentional

Select the fields your presentation layer actually uses. This keeps the Inspector easier to review and the data contract explicit.

INSPECTOR SECTION

3. Custom Field Definitions

Adds project-specific presentation metadata without modifying Upgrade SO.



Custom fields provide typed metadata with explicit Small, Big or Both scopes.

FIELD TYPE	TYPICAL PURPOSE	EXAMPLE
String	Labels and descriptions	Increases attack speed
Int / Float	Additional numeric metadata	+15 or 1.25
Color	Rarity and visual state	Legendary accent
Sprite	Icons and badges	Tier emblem
Material	Presentation material	Unlocked appearance
Texture2D	Large art or previews	Upgrade artwork

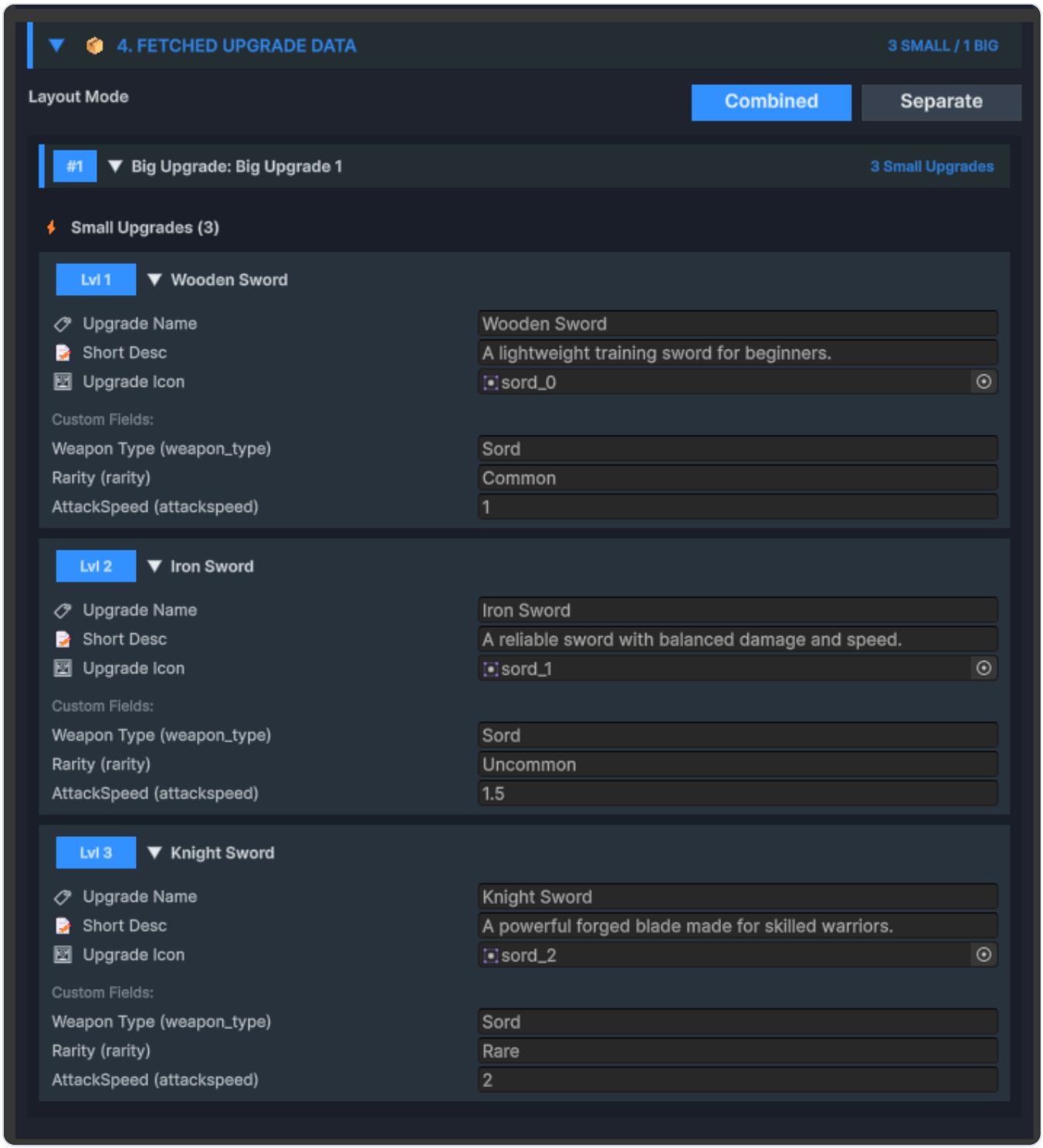
Information: Scope controls placement

SmallOnly stores a value for individual steps, BigOnly stores a group-level value, and Both makes the definition available at both levels.

INSPECTOR SECTION

4. Fetched Upgrade Data

Shows the synchronized presentation records and their custom values.



Review the fetched output to verify indices, selected fields and custom metadata before runtime.

> **Critical Requirement: Treat names and indices as authored contracts**
> If runtime code queries by name or index, keep those identifiers stable or update consumers alongside the data.

Runtime queries

Read the current, next or specifically indexed presentation record.

- GetNextUpgradeInfo

- GetUpgradeInfo by global or Big/Small index
- GetAllInfoForBig and GetBigUpgradeInfo
- GetCustomField and GetCustomFieldByIndex overloads
- CheckDirty for synchronization monitoring

CSHARP

```
UpgradeInfoProvider.SmallUpgradeInfo next =
+    infoProvider.GetNextUpgradeInfo();

if (next != null)
{
    UpgradeInfoProvider.CustomFieldValue label =
+        infoProvider.GetCustomField(next, "display-name-tag");

    Debug.Log(label.stringValue);
}
```

MODULE 07 CORE COMPONENTS

Listeners & Events

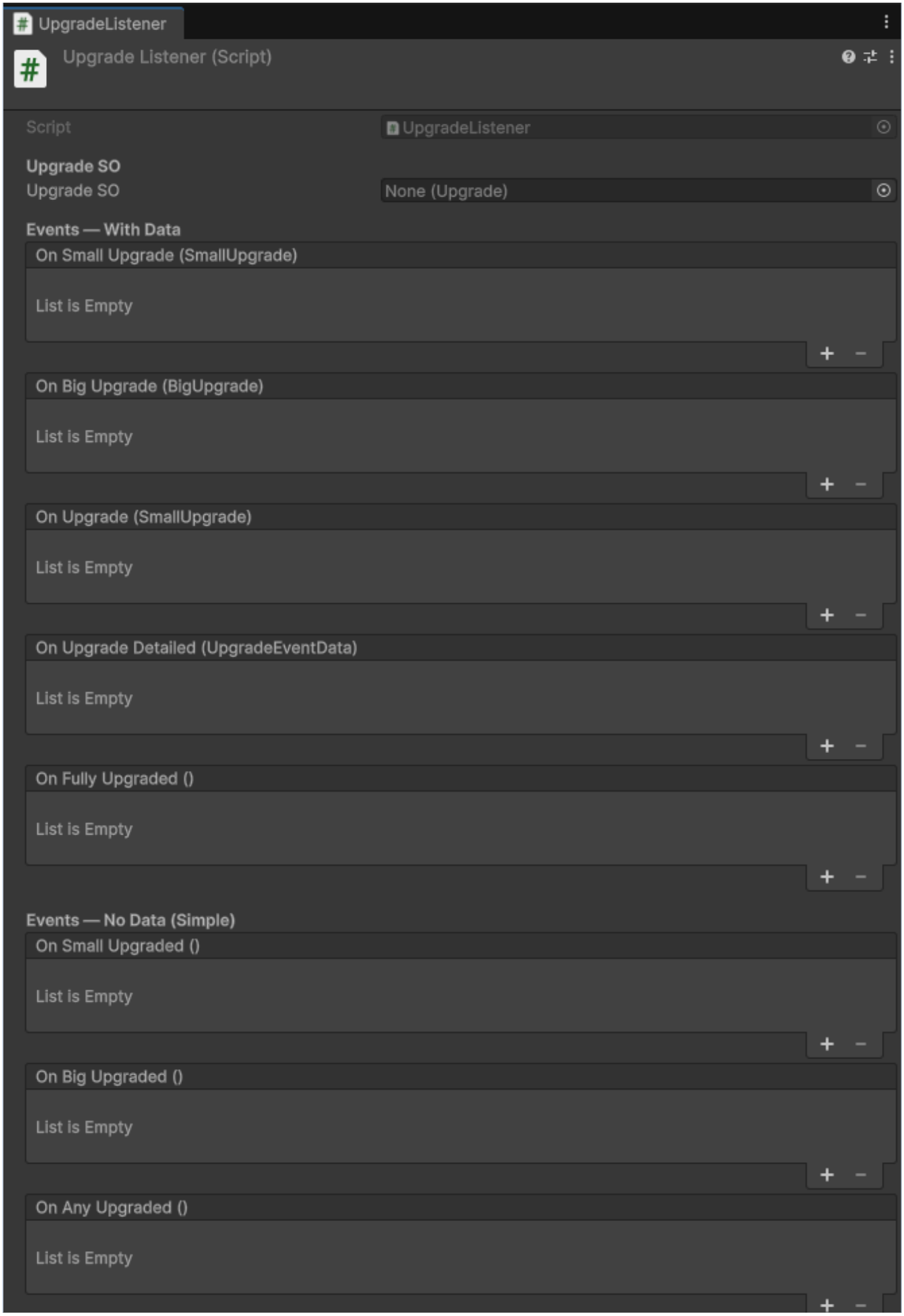
UPFS exposes C# events on its data assets and listener components for Inspector-driven UnityEvent workflows. Use either approach independently or combine them according to the architecture of your project.

COMPONENT

Upgrade Listener

Routes Upgrade SO progression events from a referenced asset into a scene object.




UpgradeListener
Upgrade Listener (Script)
Script
UpgradeListener
Upgrade SO
Upgrade SO
None (Upgrade)
Events — With Data
On Small Upgrade (SmallUpgrade)
List is Empty
On Big Upgrade (BigUpgrade)
List is Empty
On Upgrade (SmallUpgrade)
List is Empty
On Upgrade Detailed (UpgradeEventData)
List is Empty
On Fully Upgraded ()
List is Empty
Events — No Data (Simple)
On Small Upgraded ()
List is Empty
On Big Upgraded ()
List is Empty
On Any Upgraded ()
List is Empty



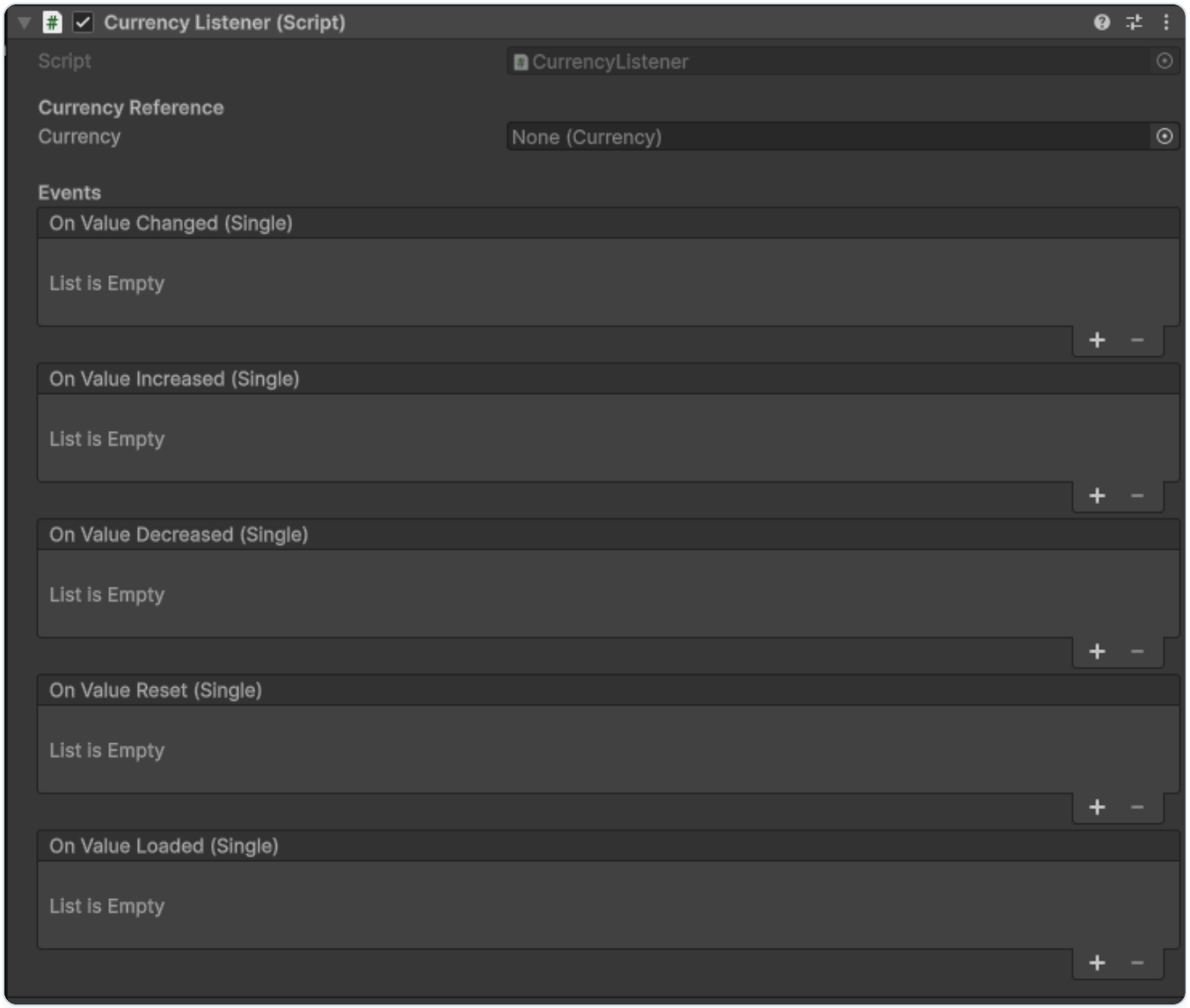
Reference one Upgrade SO and configure the UnityEvents required by this scene object.

- Respond to any successful upgrade
- Receive detailed Big and Small Upgrade context
- React to Small or Big milestone timing
- Handle the fully upgraded state

COMPONENT

Currency Listener

Routes balance changes, gains, spending, resets and loads into UnityEvents.



Currency Listener keeps balance-driven presentation logic configurable in the Inspector.

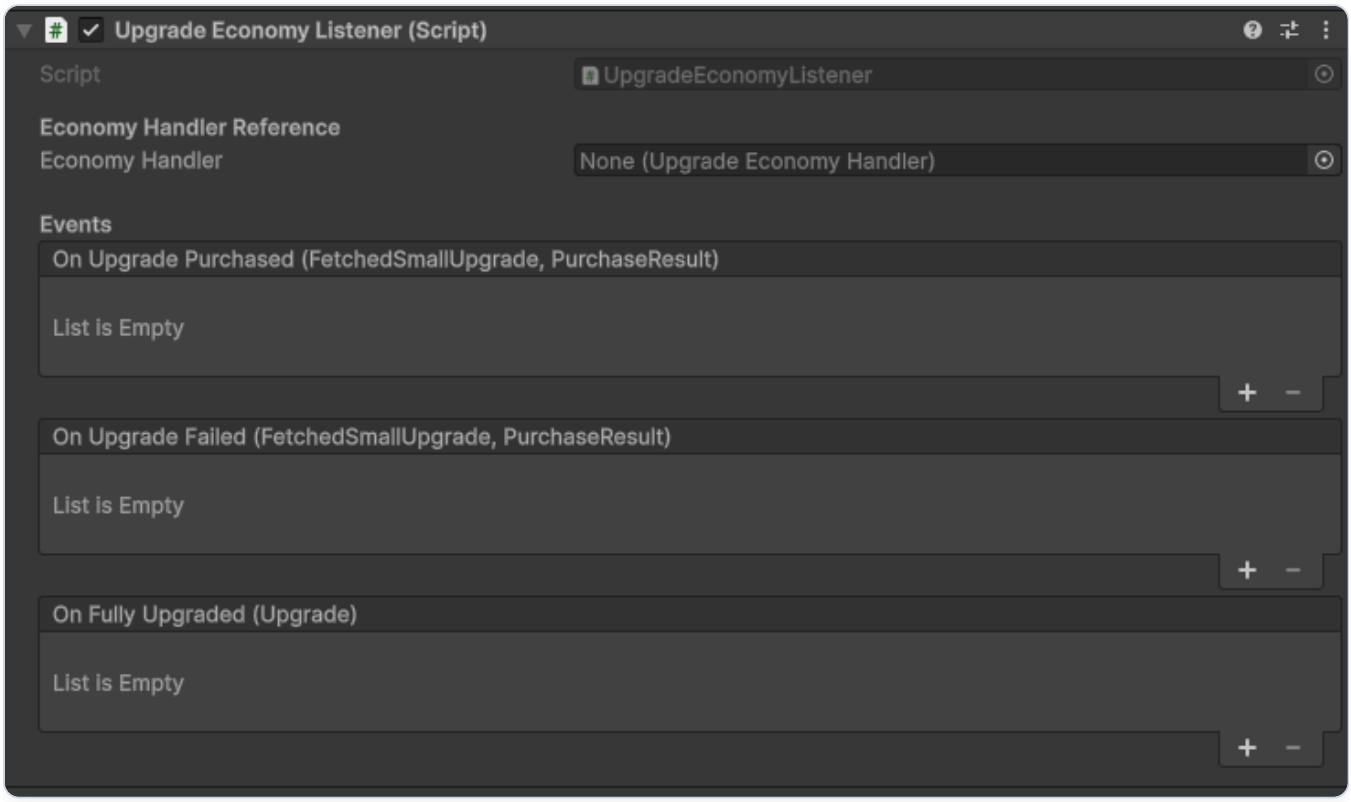
Best Practice Tip: Good UI fit

Use OnValueChanged for balance labels and OnValueDecreased for spend feedback such as particles, sounds or animations.

COMPONENT

Economy Listener

Routes successful purchases, failed attempts and full completion events.



Economy Listener provides purchase-specific feedback without coupling UI objects to purchase code.

Subscribe from code

Use C# subscriptions when handlers need typed data or should remain outside scene wiring.

CSHARP

```
private void OnEnable()
{
    gold.OnValueChanged += RefreshBalance;
    economy.OnUpgradePurchased += HandlePurchase;
}

private void OnDisable()
{
    gold.OnValueChanged -= RefreshBalance;
    economy.OnUpgradePurchased -= HandlePurchase;
}
```

Critical Requirement: Always unsubscribe

Pair subscriptions and removals in matching lifecycle methods to avoid duplicate callbacks or references to destroyed objects.

Upgrade event order

01 **State changes**

The next Small Upgrade is marked complete.

02 **General events**

Any Upgrade and detailed context events are raised.

03 **Milestone events**

Small or Big Upgrade events fire according to the configured trigger mode.

04 **Completion**

Fully Upgraded fires when no incomplete step remains.

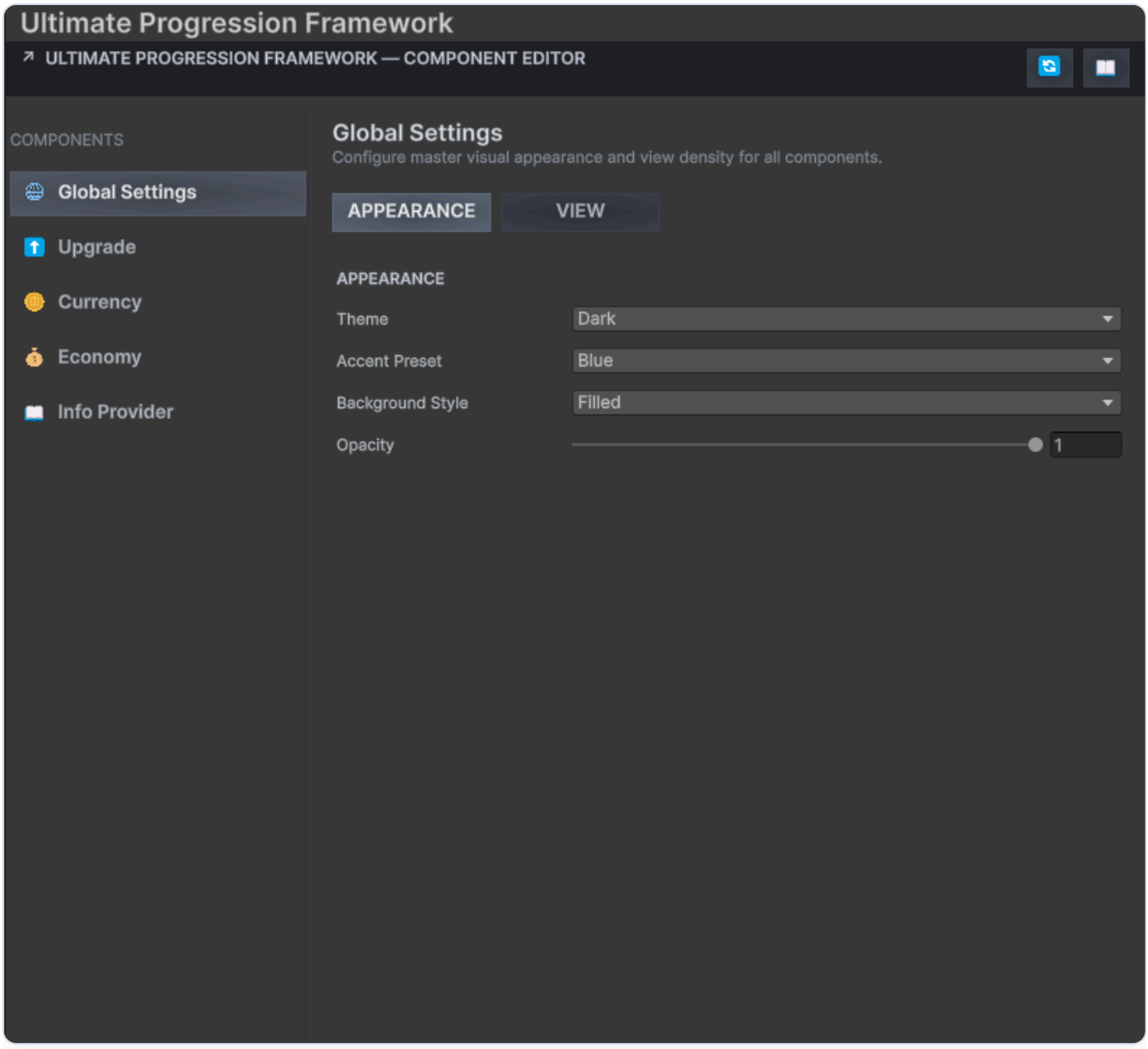
05 **Persistence**

The updated Upgrade state is saved.

MODULE 08 EDITOR & WORKFLOWS

Framework Settings

Framework Settings centralizes Editor preferences for Upgrade, Currency, Economy and Info Provider workflows. Use the global controls as your baseline, then review the component-specific options that affect authoring and synchronization.

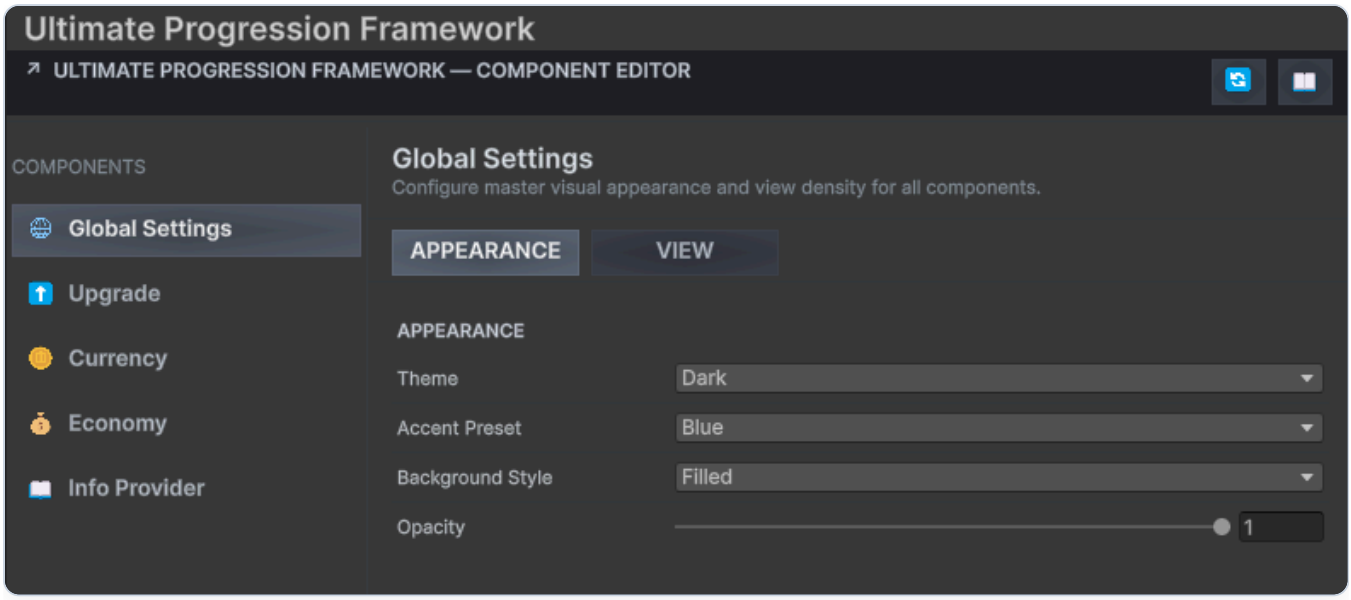


Framework Settings keeps package-wide and component-specific Editor preferences in one place.

SETTINGS SECTION

1. Global Settings

Controls shared Inspector behavior, confirmations and framework-wide authoring preferences.



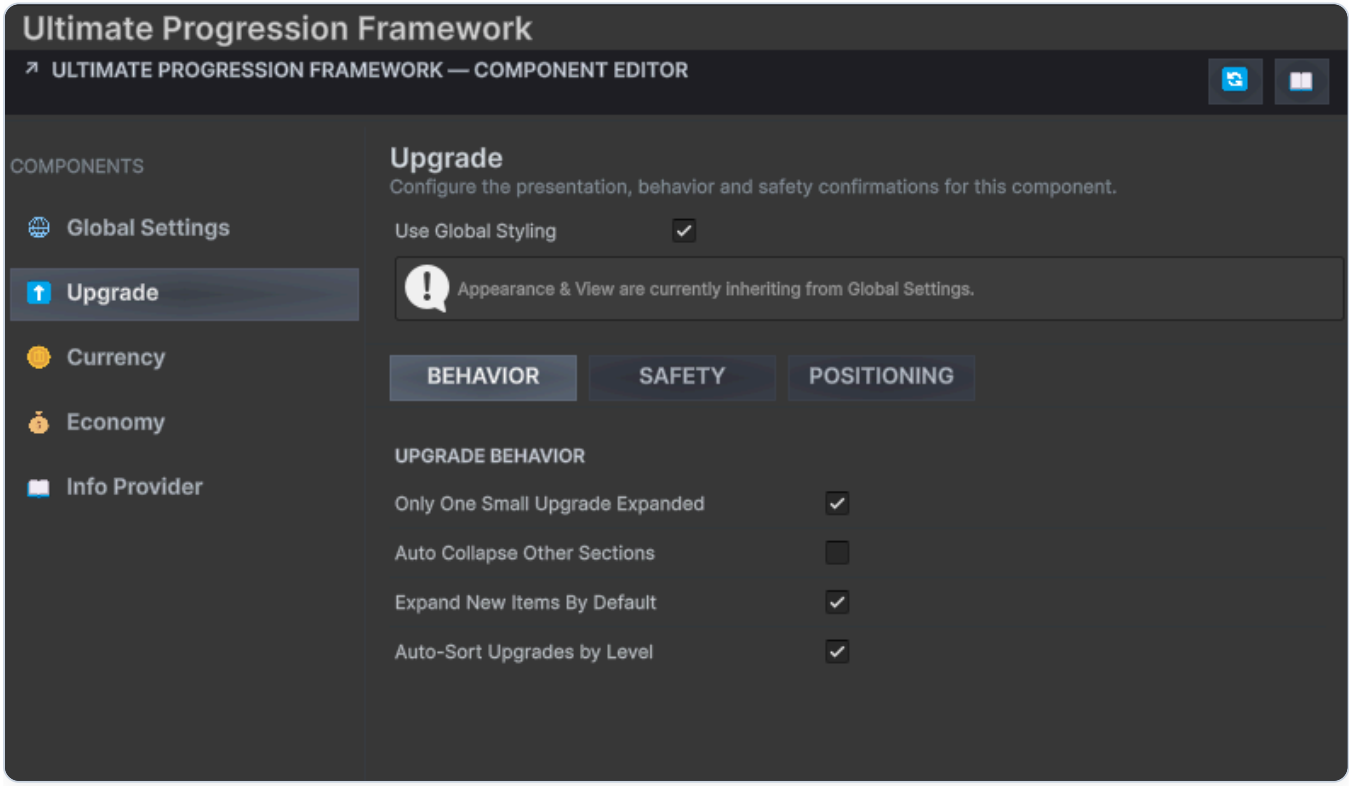
Global Settings establishes consistent behavior across the UPFS Editor experience.

> **Best Practice Tip: Team consistency**
>
> Agree on settings that alter automatic behavior before several team members begin authoring shared progression assets.

SETTINGS SECTION

2. Upgrade Settings

Configures Upgrade Inspector conveniences, generation behavior and development safeguards.

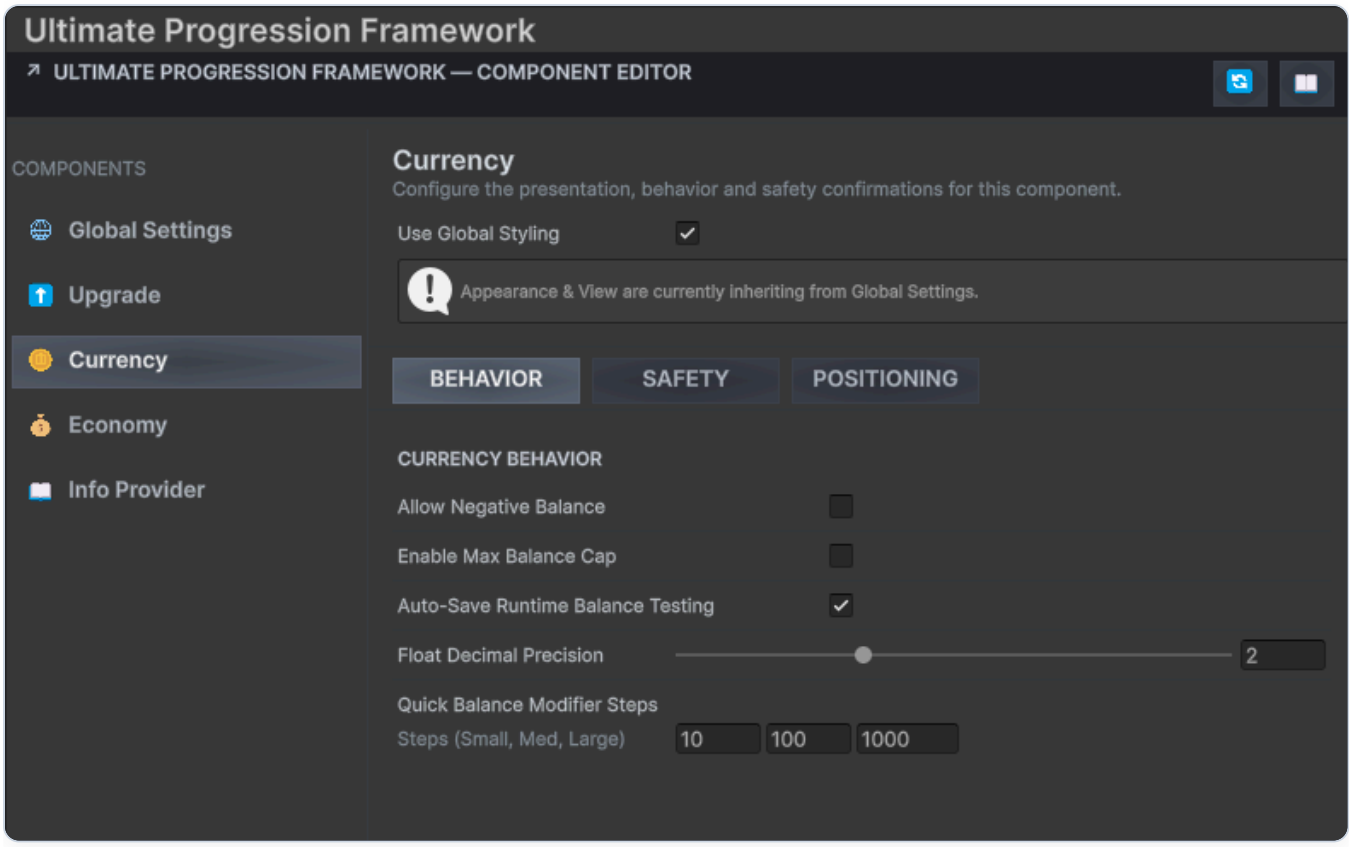


Upgrade preferences shape the main progression-authoring workflow.

SETTINGS SECTION

3. Currency Settings

Controls Currency SO Inspector and persistence-development preferences.

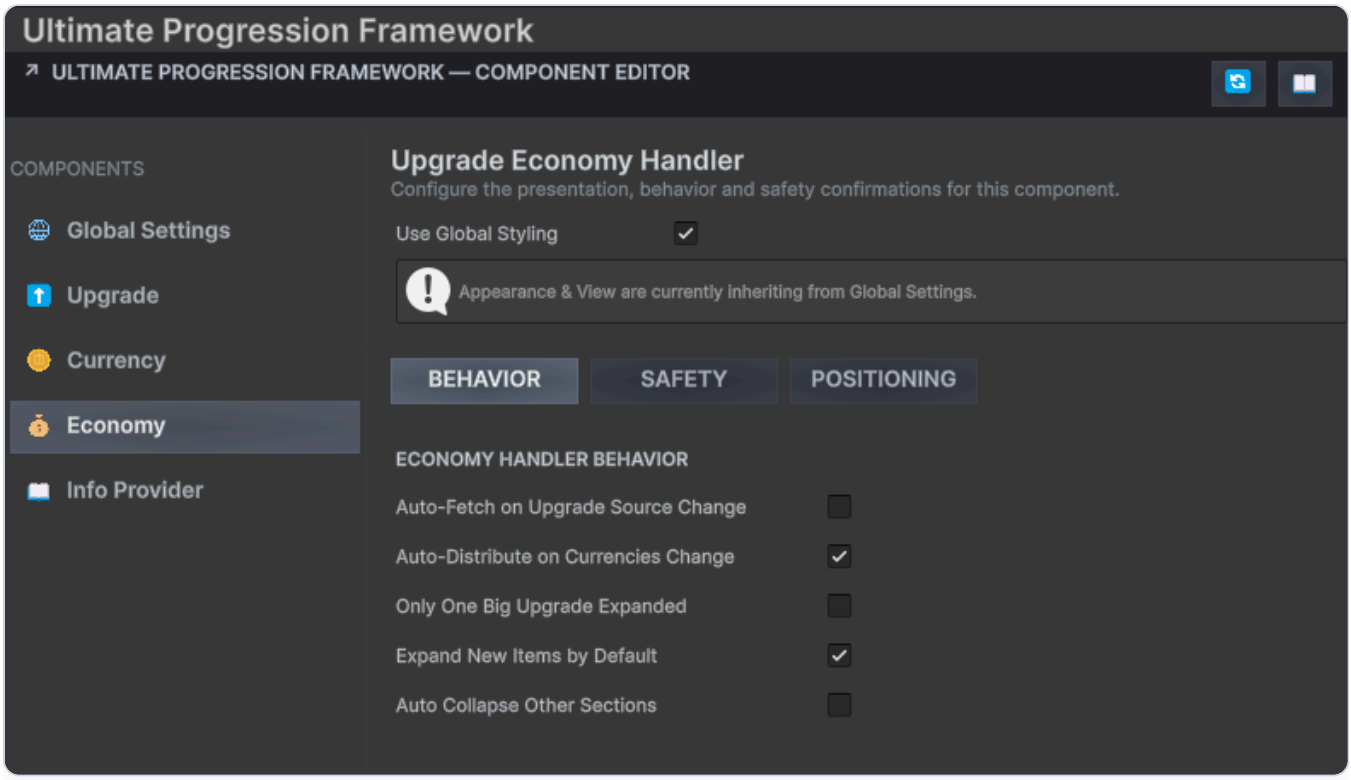


Currency preferences support predictable balance editing and testing.

SETTINGS SECTION

4. Economy Settings

Configures Economy Handler fetching, data distribution and synchronization behavior.



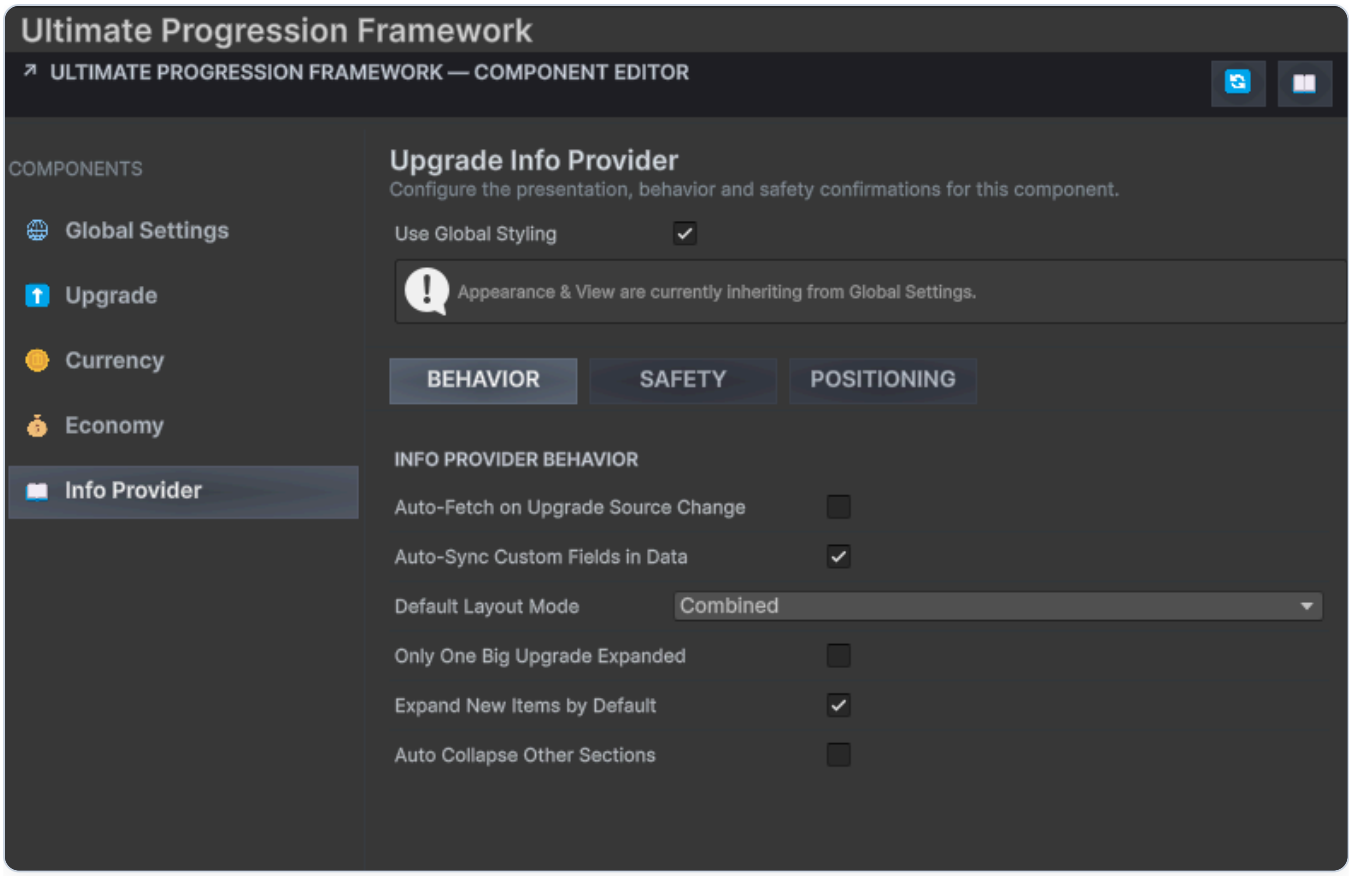
Economy preferences determine how aggressively generated cost data follows Upgrade source edits.

> **Important Notice / Warning: Automatic fetches can replace generated output**
> When enabling automatic synchronization, verify custom cost edits after structural changes to the source Upgrade.

SETTINGS SECTION

5. Info Provider Settings

Controls Info Provider synchronization and custom metadata authoring behavior.



Info Provider preferences support safe synchronization of presentation metadata.

MODULE 09 EDITOR & WORKFLOWS

Saving & Runtime State

UPFS includes immediate PlayerPrefs persistence for Upgrade and Currency assets. This page explains what is stored, when saves occur and which authoring decisions must remain stable after release.

Storage model

Upgrade SO and Currency SO persist independently under keys derived from their authored identities.

ASSET	STORED DATA	AUTOMATIC SAVE POINTS
Upgrade SO	Serialized Big Upgrade structure and Small Upgrade completion state	Successful PerformUpgrade
Currency SO	Current Int or Float balance	SetValue, ModifyValue and ResetToDefault
Economy Handler	No independent wallet save	Uses the referenced Currency and Upgrade saves
Info Provider	Authored presentation data in the asset	Not a player progression save

Critical Requirement: PlayerPrefs is the included backend

Cloud saves, encryption and external save-system adapters are not built into the current core package.

Recommended runtime lifecycle

01 **Author stable assets**

Finalize unique Upgrade and Currency identities before release.

02 **Load**

Load saved values at the initialization point defined by your project.

03 **Read and modify**

Use public APIs instead of directly editing internal serialized state.

04 **Save automatically**

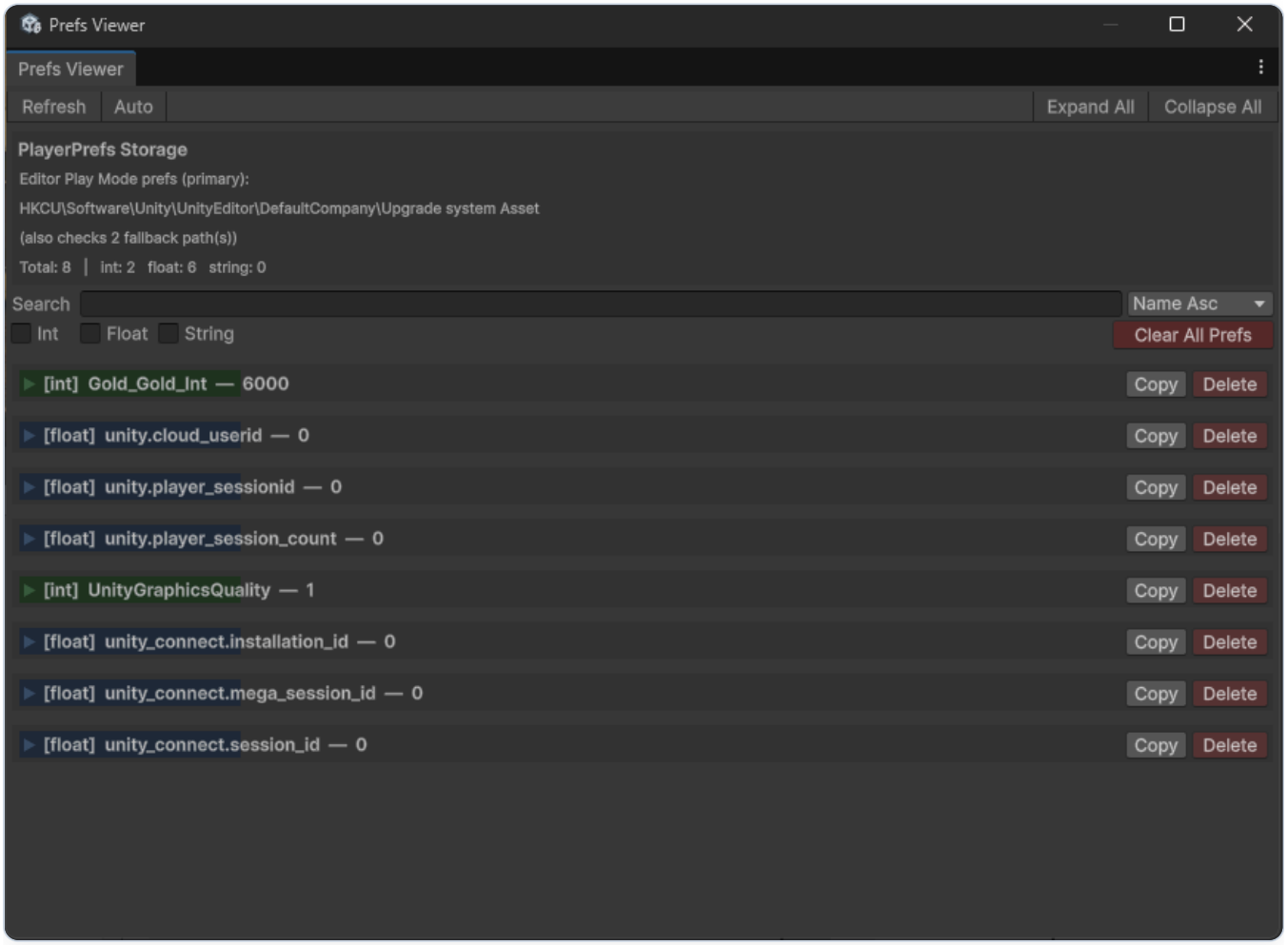
Standard progression and balance mutations save immediately.

05 **Reset intentionally**

Expose reset actions only in controlled development or player-reset flows.

PlayerPrefs Viewer

Inspect and validate stored values during development.



Use the included Editor viewer to inspect development saves without building temporary debug UI.

> **Important Notice / Warning: Clearing PlayerPrefs affects the project**
> Review every key before clearing stored data. Other systems in the same project may also use PlayerPrefs.

Renaming and migration

Asset names and configured names can participate in save keys and economy matching.

- Do not rename a released Upgrade Name without migrating its stored JSON to the new key.
- Do not rename a released Currency Name without migrating its typed PlayerPrefs entry.
- Changing Currency type from Int to Float or Float to Int should be treated as a migration.
- Keep Economy currency definition names synchronized with Wallet Currency names.

Integrating another save system

Use UPFS public state and methods as the boundary for a project-specific persistence adapter.

For projects that require cloud synchronization, profiles or encrypted files, centralize the integration in a dedicated adapter rather than spreading persistence logic across UI and gameplay scripts.

Define ownership clearly: either let UPFS PlayerPrefs remain authoritative or make the external save layer authoritative and coordinate loading before gameplay begins.

> **Information: Avoid competing authorities**
> Two save backends writing the same progression at different times can restore stale state. Establish one initialization and synchronization order.

MODULE 10 EDITOR & WORKFLOWS

Connecting Components

UPFS components are deliberately modular. Upgrade SO remains the source of progression truth, while Economy, Currency, Info Provider and listeners add only the responsibilities your feature requires.

Component responsibilities

COMPONENT	OWNS	REFERENCES
Upgrade SO	Structure, values, completion and progression events	No supporting component required
Currency SO	One shared numeric balance and its persistence	Independent data asset
Economy Handler	Costs, wallet policy, affordability and purchase transactions	Upgrade SO and Currency assets
Info Provider	Presentation-ready standard and custom metadata	Upgrade SO
Listeners	Scene-level UnityEvent routing	The asset they observe

Critical Requirement: Upgrade SO stays central

Supporting components read or extend progression behavior; they do not replace the Upgrade SO as the authoritative progression source.

Minimal progression setup

Use Upgrade SO alone when gameplay controls payment or unlock conditions elsewhere.

01 **Create Upgrade SO**
Author the complete Big and Small Upgrade sequence.

02 **Read current state**
Use GetCurrentUpgrade to initialize gameplay values.

03 **Advance**
Call PerformUpgrade after your own eligibility logic succeeds.

04 **React**
Subscribe to events or add an Upgrade Listener for presentation feedback.

Progression with an economy

Add Currency and Economy Handler when UPFS should evaluate affordability and execute payment.

01 **Create currencies**
Create one stable Currency SO for every supported payment balance.

02 **Connect source**
Assign the Upgrade SO to Upgrade Economy Handler and fetch its structure.

03 **Generate costs**
Configure matching currency names, ranges and distributions.

04 **Assign wallet**
Reference the Currency assets and select a payment mode.

05 **Purchase**
Use PurchaseNextUpgrade or PurchaseWithCurrency from the runtime UI.

> **Important Notice / Warning: Match Currency names**
> Economy cost definitions resolve Wallet entries by Currency Name. A spelling or casing mismatch prevents that Currency from satisfying the matching cost.

Presentation and UI setup

Add Info Provider when the interface needs more than level, value and price.

- Use Upgrade SO for live current and next progression state.
- Use Economy Handler for costs and affordability.
- Use Info Provider for icons, labels, colors, materials and descriptions.
- Use listener components for scene-configured refreshes and feedback.

Initialization order

01 **Load Currency and Upgrade state**

Restore authoritative player data first.

02 **Check synchronized structures**

Ensure Economy and Info Provider data reflect the current authored Upgrade structure.

03 **Bind listeners**

Enable subscriptions before the user can interact.

04 **Render UI**

Read current, next, cost and custom information into the first visible state.

MODULE 11 EDITOR & WORKFLOWS

Data Generation & Distributions

UPFS separates progression structure from curve shape. Generate the Big and Small Upgrade sequence first, then tune gameplay values, native prices and economy costs with distributions that fit the intended player experience.

Recommended generation order

01 Define milestones
Choose the number of Big Upgrades and Small Upgrades per group.

02 Set gameplay ranges
Choose starting and ending values for the progression.

03 Set native price ranges
Define the Upgrade SO price range when the native price is used.

04 Generate Upgrade data
Create and review the nested progression.

05 Fetch dependents
Refetch Economy Handler and Info Provider after the structure is stable.

06 Distribute currency costs
Generate each economy series independently and review the output.

Important Notice / Warning: Generate deliberately
Regeneration can replace previously authored values. Confirm the operation and keep source control or a backup when balancing production data.

Distribution types

DISTRIBUTION	CURVE CHARACTER	BEST FIT
Linear	Constant rate of change	Transparent, predictable progression
Exponential / Ease In	Late values grow more quickly	Escalating long-term costs
Logarithmic / Ease Out	Early values grow more quickly	Fast early power gains
Ease In Out	Slow–fast–slow S-curve	Smooth, milestone-oriented pacing
Random	Independent values inside bounds	Controlled variation
Step	Flat generated output	Manually differentiated entries
Custom Curve	Authored AnimationCurve	Designer-owned progression shapes

How curve evaluation works

Each sequential Small Upgrade receives a normalized position from the beginning to the end of the complete progression.

The selected distribution transforms that normalized position before UPFS interpolates between the configured start and end values.

Integer configurations round the resulting values. Float configurations preserve decimal precision where supported.

> **Best Practice Tip: Evaluate representative points**
> During balancing, compare the first, quarter, middle, three-quarter and final values instead of inspecting only the endpoints.

Combine independent curves

Gameplay growth and purchase cost do not need to use the same distribution.

DESIGN GOAL	GAMEPLAY VALUE	COST
Accessible early progression	Ease Out	Linear
Long-term economy sink	Linear	Exponential
Milestone-focused power	Ease In Out	Ease In Out
Bespoke live balance	Custom Curve	Custom Curve

Distribution API

Upgrade SO exposes methods for applying value and price distributions to its current data.

CSHARP

```
float value = upgrade.ApplyValueDistribution(
+    normalizedPosition, startValue, endValue
+);

int price = upgrade.ApplyPriceDistribution(
+    normalizedPosition, startPrice, endPrice
+);
```

Information: Prefer Editor-time balancing

Generate and review production curves before release. Runtime redistribution should be an intentional feature, not a substitute for authored balance data.

MODULE 12 EDITOR & WORKFLOWS

Common Workflows

These focused recipes combine the core UPFS components into common production patterns. Begin with the smallest useful configuration and add supporting systems only where the feature requires them.

Upgrade button with affordability

01 **Read next state**
Use GetNextUpgrade for the next level and gameplay value.

02 **Read next cost**
Use Economy Handler to retrieve the matching currency cost.

03 **Set interactability**
Use CanAffordNextUpgrade to enable or disable the purchase button.

04 **Purchase**
Call PurchaseNextUpgrade when the player confirms.

05 **Refresh**
Respond to purchase and Currency events instead of polling every frame.

Player-selected payment currency

CSHARP

```
List<UpgradeEconomyHandler.AffordableOption> options =
+    economy.GetAffordableOptionsForNext();

// Present options in the UI, then purchase with the player's choice.
economy.PurchaseWithCurrency(selectedCurrency);
```

Critical Requirement: Use Player Choice mode
The selected Currency must exist in the Wallet, match generated cost data and have enough balance for the next upgrade.

Buy Max preview

Calculate how many sequential upgrades a balance can currently afford.

- Call GetAffordableUpgradeCount using a Currency name or ID.
- Display the returned count before the player confirms a bulk action.
- Execute purchases through the standard purchase API so balance checks, events and saves remain consistent.
- Stop when the progression is complete or the selected Currency can no longer afford the next cost.

Progress bars and milestone UI

UI ELEMENT	SOURCE
Overall bar	Upgrade.GetOverallUpgradeProgress
Current group bar	Upgrade.GetCurrentBigUpgradeProgress
Completed counter	Upgrade.GetUpgradedSmallCount / GetTotalSmallUpgrades
Next cost progress	Economy.GetProgressToNextUpgrade
Tier icon or description	Info Provider custom fields

Dynamic upgrade card

Combine three read models into one interface without duplicating authored data.

- Upgrade SO: current state, next level and gameplay value
- Economy Handler: costs, affordability and payment options
- Info Provider: title, description, icon, color and preview assets
- Listeners: event-driven refresh and feedback

MODULE 13 REFERENCE & SUPPORT

Troubleshooting & Validation

Most UPFS issues come from missing references, unstable identities or dependent data that was not refetched after a structural edit. Start with the exact validation message, then verify the relevant relationship below.

Read Inspector validation first

UPFS surfaces configuration problems near the component that needs attention.

INSPECTOR REFERENCE

Validation Messages

`images/tools/validation-messages.png`

Recommended Asset Format: 1400 × 900 PNG

Capture a representative Inspector showing clear UPFS warning and error messages.

Best Practice Tip: Fix the first upstream issue

A missing Upgrade Source can cause several downstream warnings. Resolve source and identity errors before regenerating dependent data.

Purchase fails

SYMPTOM	LIKELY CAUSE	RESOLUTION
No next upgrade	Progression is complete or contains no generated entries	Check IsFullyUpgraded and review Big Upgrades
No matching cost	Currency names differ between definition and Wallet asset	Match names exactly and refetch/distribute
Cannot afford	Balance is below the next generated cost	Inspect GetNextUpgradeCost and Currency.GetValue
Player Choice rejected	Selected Currency is unavailable or not affordable	Present GetAffordableOptionsForNext results
Stale cost/index	Upgrade structure changed after economy fetch	Fetch Upgrade Info again

State does not load as expected

- Confirm the current Upgrade Name or Currency Name matches the name used when the save was created.
- Confirm Currency Type has not changed between Int and Float.
- Use PlayerPrefs Viewer to verify that the expected key exists.
- Check whether another development tool or reset flow cleared PlayerPrefs.
- If first-as-base is enabled, remember that reset reapplies the first entry as the base state.

> **Important Notice / Warning: Do not repeatedly reset while diagnosing**
> Capture the current key and value before using reset actions. Resetting removes the evidence needed to distinguish a loading problem from a naming problem.

Info Provider data is stale

01 **Check source**
Verify that the correct Upgrade SO is assigned.

02 **Check structure**
Compare Big and Small Upgrade counts with fetched records.

03 **Refetch safely**
Use preserveExisting when custom metadata should be retained.

04 **Revalidate queries**
Confirm name and index-based consumers still point to the intended entries.

Confirmation dialogs

Destructive Editor operations request confirmation before replacing or clearing data.

INSPECTOR REFERENCE

Confirmation Dialog

`images/tools/confirmation-dialog.png`

Recommended Asset Format: 1000 × 600 PNG

Document one representative confirmation dialog used before a destructive generation or reset action.

Custom Inspector is missing

Unity must compile successfully before UPFS Editor tooling can load.

- Open Console and resolve the first compile error in the project.
- Confirm the UPFS package imported completely, including its Editor folder.
- Reimport the affected script only after resolving compilation failures.
- Verify that runtime code uses the bcylyLabs.UltimateProgressionFramework namespace.

MODULE 14 REFERENCE & SUPPORT

FAQ & Support

Use this page for quick decisions and support preparation. For component behavior, follow the linked manual concepts and API references in the documentation navigation.

Do I need every UPFS component?

No. Upgrade SO is the only central progression component.

Add Currency and Economy Handler when UPFS should own affordability and payment. Add Info Provider for presentation metadata. Add listener components when Inspector-configured UnityEvents fit the scene workflow.

Can one project contain multiple progressions?

Yes. Create a separate Upgrade SO for each independent progression.

Weapon power, storage, player rank and factory speed can each use their own Upgrade asset while sharing Currency assets where appropriate.

Should a Currency use Int or Float?

Use Int for whole units and Float only when fractional balances are meaningful to gameplay.

> **Important Notice / Warning: Choose before release**
> Changing the numeric type later changes persistence behavior and should be handled as a save migration.

Does UPFS include cloud saving?

No. The included persistence uses PlayerPrefs.

Projects that require profiles, encryption, platform cloud storage or a third-party save framework should integrate those requirements through a project-owned adapter and a defined initialization order.

Is UPFS tied to a render pipeline?

No. Core UPFS is data, runtime logic and Editor tooling and is independent of Built-in, URP and HDRP.

Can I rename assets after release?

Treat configured Upgrade and Currency names as stable data identifiers.

A visual Project-window filename change is different from changing a configured name used by persistence or economy matching. Test migrations with representative existing saves before shipping renamed identifiers.

Prepare a useful support request

- Unity version and UPFS version
- Exact Console error and full stack trace
- Screenshot of the complete relevant Inspector section
- Payment mode, Currency type and whether save data already existed
- Small reproduction steps beginning from entering Play Mode

> **Best Practice Tip: Minimal reproduction**
> A small test scene containing only the affected assets is usually the fastest way to separate framework configuration from project-specific code.

MODULE 15 API REFERENCE

Upgrade API Reference

Upgrade is the authoritative runtime API for progression state. All types below are in the bcylyLabs.UltimateProgressionFramework namespace.

Progression methods

SIGNATURE	RETURNS	BEHAVIOR
PerformUpgrade()	bool	Completes the next sequential Small Upgrade, raises events and saves
IsFullyUpgraded()	bool	True when every Small Upgrade is complete
HasAnyUpgrade()	bool	True when at least one upgrade entry exists
ApplyFirstUpgradeAsBaseIfNeeded()	void	Applies the first entry silently when first-as-base is enabled

State and query methods

SIGNATURE	RETURNS	PURPOSE
GetCurrentUpgrade()	SmallUpgrade	Current completed state or configured base value
GetNextUpgrade()	SmallUpgrade	First incomplete sequential entry
GetNextUpgradeCost()	int	Native price of the next entry
GetNextUpgradeValue()	float	Gameplay value of the next entry
GetSmallUpgradeByIndex(int globalIndex)	SmallUpgrade	Entry at the zero-based flattened index
GetUpgradedSmallCount()	int	Number of completed sequential entries
GetTotalSmallUpgrades()	int	Total entries across all Big Upgrades
GetCurrentBigUpgradeProgress()	float	Normalized progress inside the current Big Upgrade
GetOverallUpgradeProgress()	float	Normalized progress across the complete asset

Persistence methods

SIGNATURE	BEHAVIOR
SaveDataFunction()	Serializes and writes current Big Upgrade data to PlayerPrefs
LoadData()	Loads and replaces state from the current save key
ResetSave()	Deletes the key, clears state and reapplies the optional first base entry
OnPlayerPrefsCleared()	Handles state when PlayerPrefs has been cleared according to configuration

Distribution methods

SIGNATURE	RETURNS	PURPOSE
ApplyValueDistribution(float t, float start, float end)	float	Evaluates the selected value distribution at normalized position t
ApplyPriceDistribution(float t, int start, int end)	int	Evaluates and rounds the selected price distribution

Events

EVENT	PAYLOAD	RAISED WHEN
onUpgrade	SmallUpgrade	A Small Upgrade completes
onUpgradeDetailed	UpgradeEventData	A step completes with Big, Small and index context
onSmallUpgrade	SmallUpgrade	The configured Small Upgrade event point occurs
onBigUpgrade	BigUpgrade	The configured Big Upgrade milestone occurs
onFullyUpgraded	—	No incomplete entry remains
onAnyUpgraded	—	Any standard upgrade succeeds
onSmallUpgraded	—	A Small Upgrade event is raised
onBigUpgraded	—	A Big Upgrade event is raised

Core data types

TYPE	KEY FIELDS
SmallUpgrade	isUpgraded, upgradeLevel, upgradeValue, upgradePrice
BigUpgrade	bigUpgradeName, smallUpgradesCount, upgradeData
UpgradeEventData	smallUpgrade, bigUpgrade, bigIndex, smallIndex, globalSmallIndex
BaseUpgradeValues	upgradeLevel, upgradeValue, upgradePrice

Important Notice / Warning: Do not mutate serialized fields casually

Prefer public Upgrade methods for runtime progression. Direct list edits can bypass event, sequencing and persistence guarantees.

MODULE 16 API REFERENCE

Currency API Reference

Currency provides a small, predictable API for a shared typed balance. Set and modify operations apply Int or Float behavior, save immediately and raise focused events.

Public configuration and state

MEMBER	TYPE	PURPOSE
id	int	Unique lookup identifier
currencyName	string	Display, persistence and economy-matching name
type	CurrencyType	Int or Float behavior
defaultValue	float	First-run and reset value
currentValue	float	Current serialized runtime balance

Balance methods

SIGNATURE	RETURNS	BEHAVIOR
GetValue()	float	Returns the current typed balance
SetValue(float newValue)	void	Replaces, saves and raises change-direction events
ModifyValue(float amount)	void	Adds a positive or negative delta, then saves and raises events
HasEnough(float amount)	bool	Checks whether the current balance meets the amount

Persistence methods

SIGNATURE	BEHAVIOR
SaveData()	Writes the current typed value to PlayerPrefs
LoadData()	Loads the typed key or uses Default Value
ResetToDefault()	Restores Default Value, saves and raises reset/change events

Events

EVENT	PAYLOAD MEANING
OnValueChanged	New current balance
OnValueIncreased	Positive amount gained
OnValueDecreased	Positive magnitude deducted
OnValueReset	Balance after reset
OnValueLoaded	Balance after load

Safe purchase guard

CSHARP

```
const float cost = 125f;

if (gold.HasEnough(cost))
{
    gold.ModifyValue(-cost);
}
else
{
    Debug.Log("Not enough Gold");
}
```

Best Practice Tip: Prefer Economy Handler for upgrade purchases

Use direct Currency mutations for general gameplay transactions. Economy Handler coordinates the balance deduction and Upgrade advancement as one progression purchase flow.

MODULE 17 API REFERENCE

Economy & Info API Reference

Upgrade Economy Handler and Upgrade Info Provider are optional read-and-action layers around Upgrade SO. Economy owns payment decisions; Info Provider owns UI-oriented metadata.

Economy synchronization

SIGNATURE	BEHAVIOR
FetchUpgradeInfo()	Rebuilds flattened economy entries from the assigned Upgrade SO
DistributeAllCurrencies()	Generates every configured currency cost series
IsFullyUpgraded()	Delegates completion state to the linked Upgrade

Cost and affordability queries

SIGNATURE	RETURNS
GetNextUpgradeCost(string currencyName)	Next cost for a named currency
GetNextUpgradeCost(int currencyId)	Next cost for a Wallet Currency ID
GetUpgradeCost(int globalIndex, string currencyName)	Named cost at a flattened index
GetUpgradeCost(int bigIndex, int smallIndex, string currencyName)	Named cost at nested indices
GetUpgradeCost(..., int currencyId)	Equivalent ID-based overloads
CanAffordNextUpgrade()	Whether the current payment mode can pay for the next step
CanAffordUpgrade(int globalIndex)	Whether any valid Wallet option can pay
CanAffordUpgrade(int globalIndex, Currency chosenCurrency)	Whether the supplied Currency can pay

Selection, progress and Buy Max

SIGNATURE	RETURNS
GetAffordableOptionsForNext()	List for Player Choice UI
GetPriorityAffordableCurrencyIndexForNext()	Wallet index of the first priority option
GetPriorityAffordableCurrencyForNext()	First priority Currency that can pay
GetProgressToNextUpgrade(string currencyName)	Normalized balance progress toward next cost
GetProgressToNextUpgrade(int currencyId)	ID-based equivalent
GetAffordableUpgradeCount(string currencyName)	Sequential upgrade count affordable by name
GetAffordableUpgradeCount(int currencyId)	Sequential upgrade count affordable by ID
GetWalletCurrencyById(int currencyId)	Matching Wallet Currency or no match

Purchase methods and results

PurchaseResult

Every purchase returns a structured result.

- success
- failReason
- currencyUsed
- amountDeducted

Economy events

Subscribe for transaction feedback and completion.

- OnUpgradePurchased
- OnUpgradeFailed
- OnFullyUpgraded

SIGNATURE	BEHAVIOR
PurchaseNextUpgrade()	Selects payment according to Payment Mode and attempts the next purchase
PurchaseWithCurrency(Currency chosenCurrency)	Attempts the next purchase using the supplied Wallet Currency
PurchaseWithCurrency(int currencyID)	ID-based selected-currency overload

Info Provider synchronization and lookup

SIGNATURE	RETURNS / BEHAVIOR
FetchUpgradeInfo(bool preserveExisting)	Rebuilds records and optionally preserves authored values
GetNextUpgradeInfo()	SmallUpgradeInfo for the next progression step
GetUpgradeInfo(int globalIndex)	SmallUpgradeInfo at flattened index
GetUpgradeInfo(int bigIndex, int smallIndex)	SmallUpgradeInfo at nested indices
GetAllInfoForBig(int bigIndex)	All SmallUpgradeInfo records in one group
GetBigUpgradeInfo(int bigIndex)	BigUpgradeInfo for one group
CheckDirty()	Checks whether source synchronization is stale

Custom field lookup

SIGNATURE	LOOKUP
GetCustomField(SmallUpgradeInfo info, string tag)	Small field by stable tag
GetCustomField(BigUpgradeInfo info, string tag)	Big field by stable tag
GetCustomField(int globalIndex, string tag)	Small field at flattened index
GetCustomFieldByIndex(SmallUpgradeInfo info, int fieldIndex)	Small field by definition order
GetCustomFieldByIndex(BigUpgradeInfo info, int fieldIndex)	Big field by definition order

Best Practice Tip: Prefer stable tags

Tag-based lookup is clearer and more resilient than field-index lookup when custom definitions may be reordered during development.